Department of CSE

Computer Graphics

Unit 1

Introduction to Computer Graphics

By

Dr. Srividya M S

Objectives

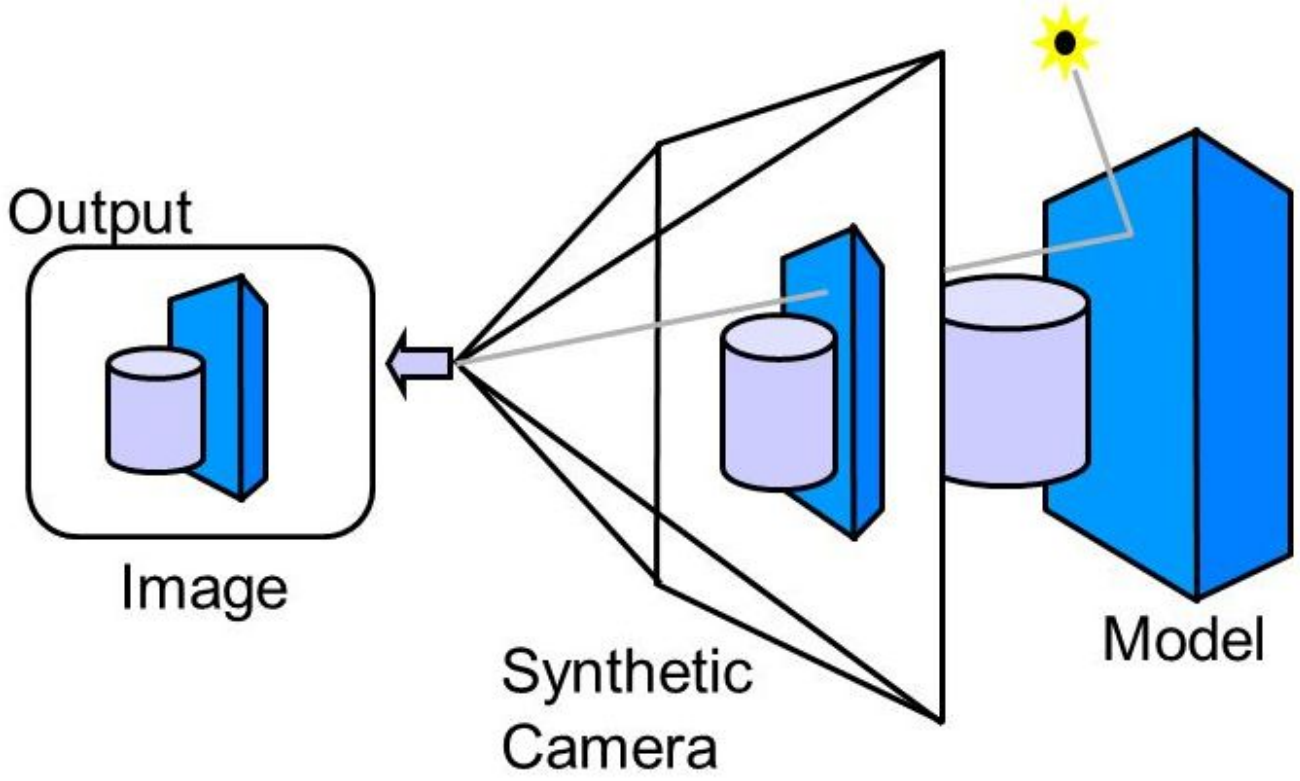
- Explanation of the basic graphics system.
- Explanation of Imaging systems: Physical and synthetic.
- Working of a synthetic camera model.
- Basic idea of the programmer's interface.
- Learning the basic design of a graphics system.
- Introducing pipeline architecture.
- Generation of the ***Sierpinski gasket.***

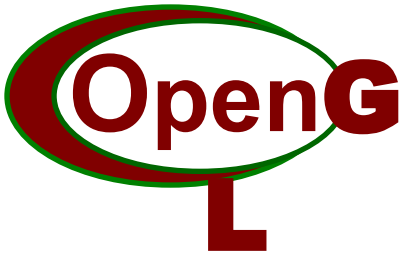
Computer Graphics

Computer Graphics

- Computer Graphics deals with all aspects of ***creating images*** with a computer.
- Computer Graphics deals with
  - **Geometric modeling:** creating mathematical models of 2D and 3D objects.
  - **Rendering:** producing images of the objects.
  - **Animation:** representing time dependent behaviour of objects.
- Computer Graphics software – ***OpenGL.***

Computer Graphics

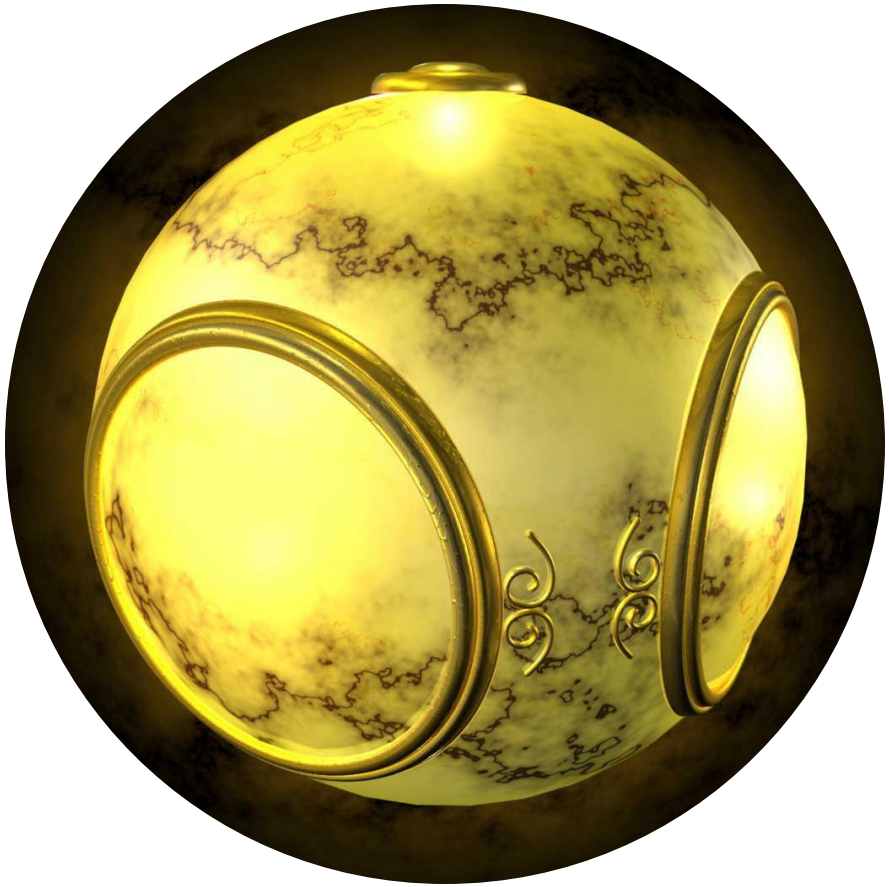




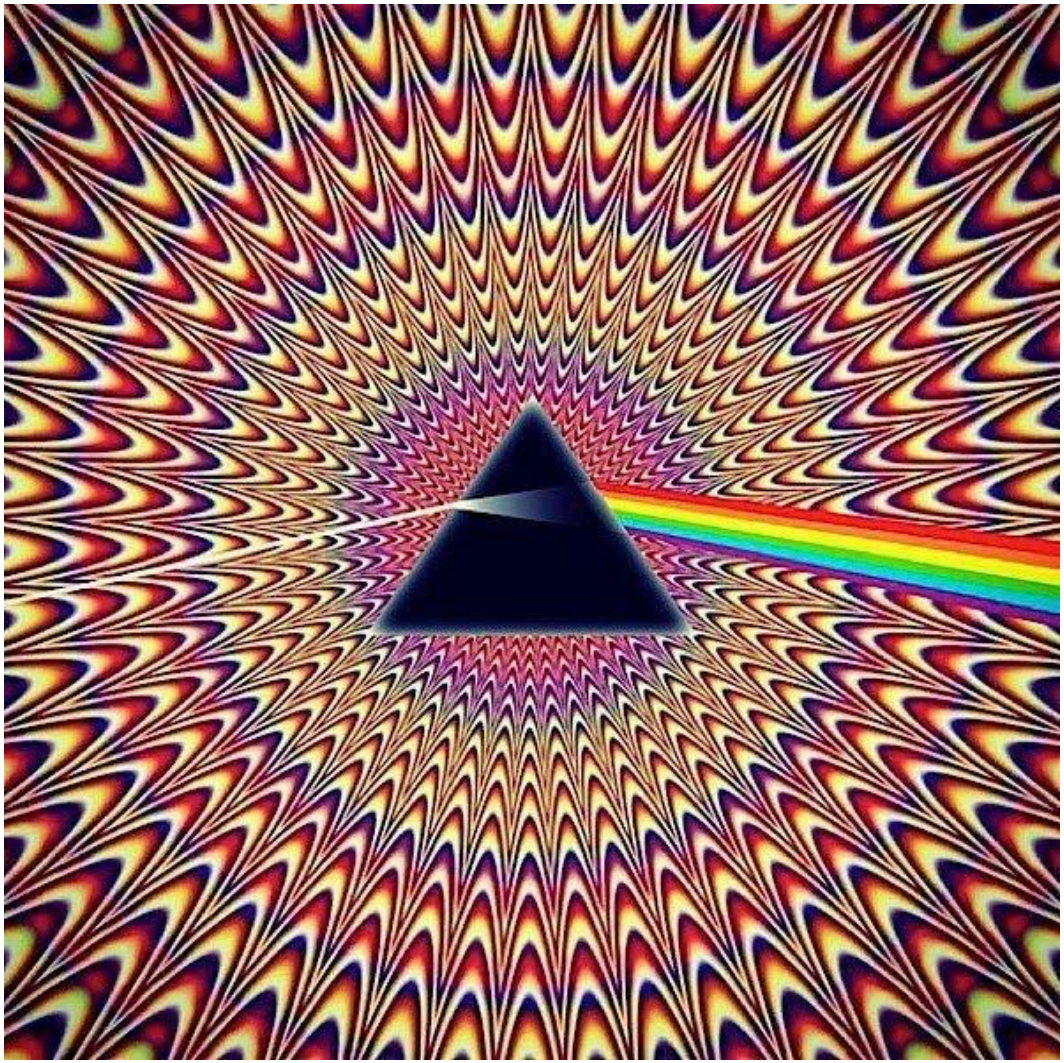
Open Graphics Library

- ***Tool or a method*** used to develop graphics application.
- Easy to learn.
- Popular graphics system.
- Follows top-down approach.

Computer Graphics - Example

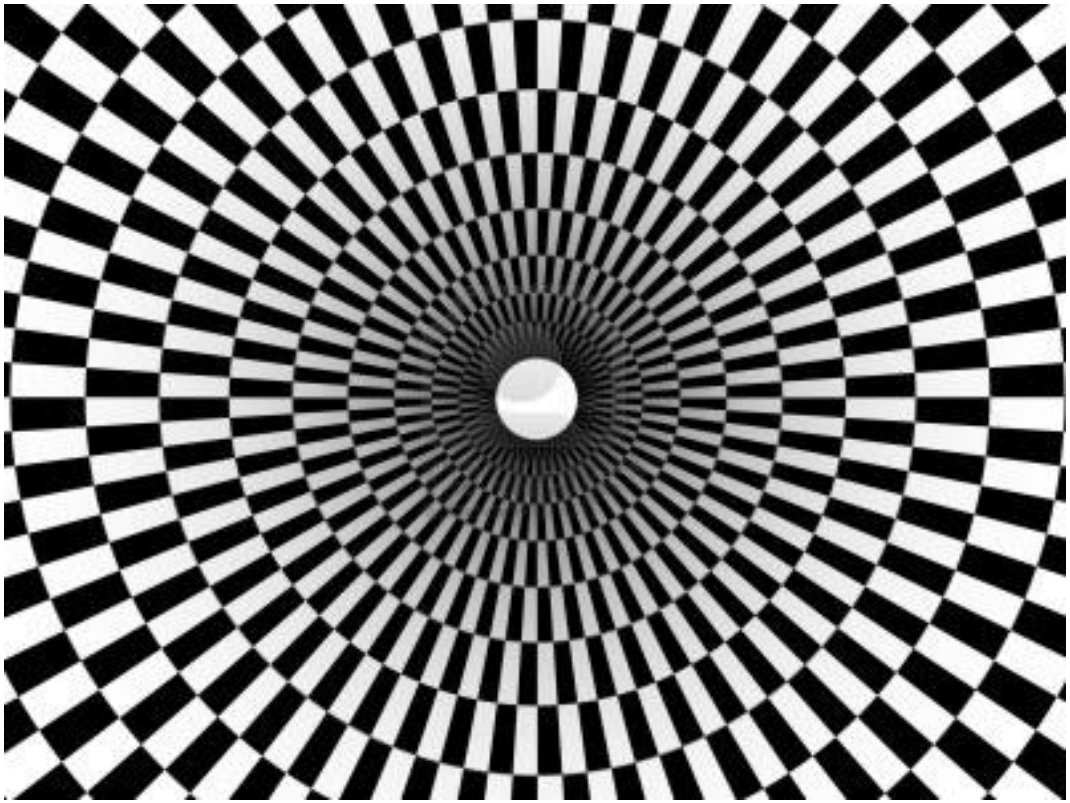


Computer Graphics - Example



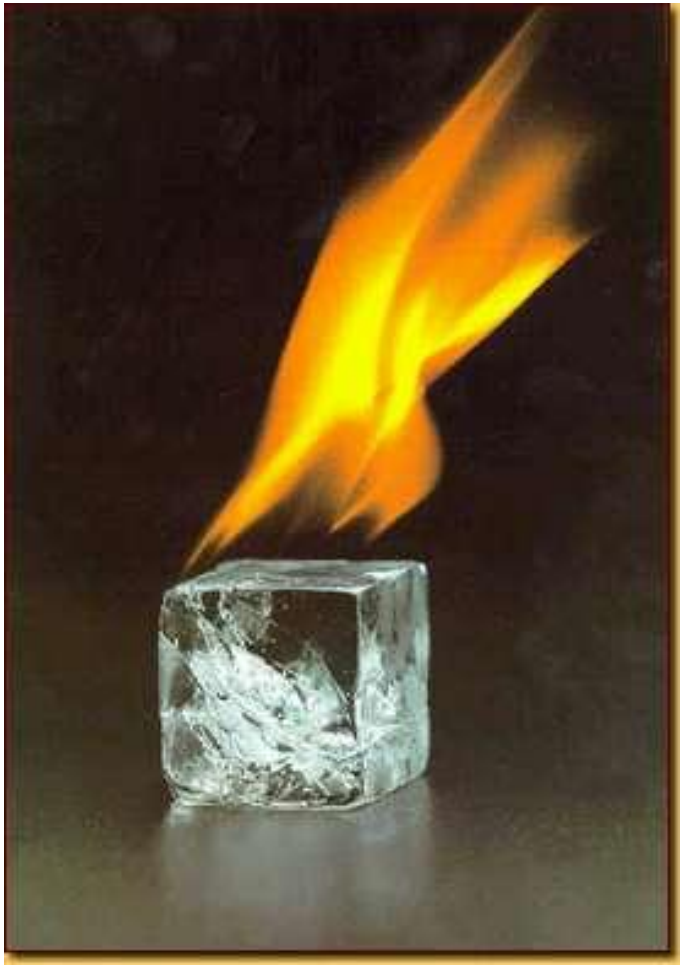
Graphics

Computer Graphics - Example



Graphics

Graphics



Applications of Computer Graph

Applications of Computer Graphics

- Display of information
- Design
- Simulation and animation
- User interfaces

Display of information

- Graphical *medium* to convey information among people.
- More than 4000 yrs ago, Babylonians displayed floor plans of buildings on stones.
- More than 2000 yrs ago, Greeks conveyed their architectural ideas graphically.

Display of information

- Maps can be developed and manipulated to display geographical and celestial information over the world.

Display of information

- Medical imaging – CT scan, MRI scan, PET scan etc generate 3-D data & is subjected to algorithmic manipulation to provide useful information.
- Fluid flow, molecular biology and mathematics.

Design

- Interactive graphics tools are extensively used in the ***field of architecture, mechanical engineering.***
- CG is used in the design of VLSI circuits, creation of ***characters for animations.***

Simulation and animation

- Once graphics system evolved to be capable of generating sophisticated images in real time, engineers and researchers began to use them as simulators.
- **Training of pilots (video)**
  - A flight simulator is a device that artificially re-creates aircraft flight and the environment in which it flies, for pilot training, design, or other purposes.
  - Training expenses have been reduced with increased safety.

Simulation and animation

Training of pilots (video)

Simulation and animation

- Designing robots, planning its path, simulating its behavior in complex environments.
- Animation in television, motion pictures and advertising industries.

Virtual Environment !

Massive multiplayer games

User interfaces

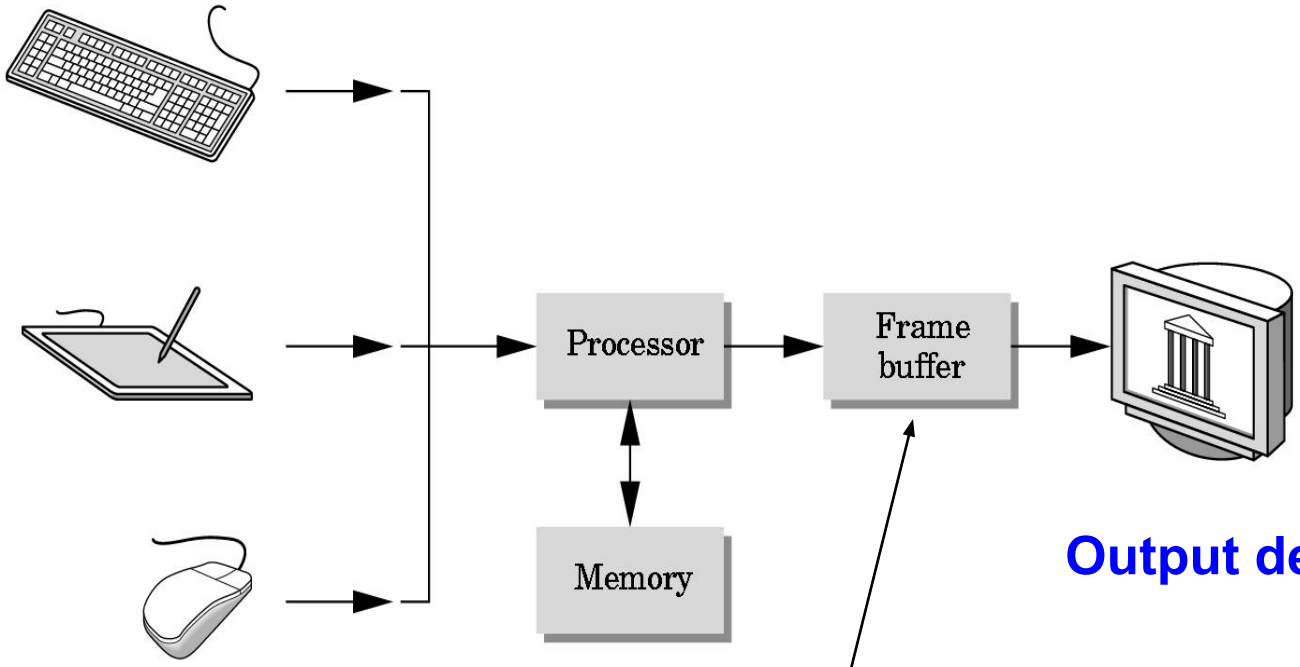
- In the field of computers, ***user interaction system*** is dominated by windows, icons, menus and pointing devices like mouse.
- This style of interface makes wide use of CG.
- GUI is one of the most popular interfaces today.

Graphics System

A Graphics System

- A graphics system is specialized for ***performing graphic based operations*** and ***image generations***.
- Has all components of a general-purpose computer.
- 5 major elements
  - **Input devices**
  - **Processor**
  - **Memory**
  - **Frame buffer**
  - **Output devices**

Basic Graphics System

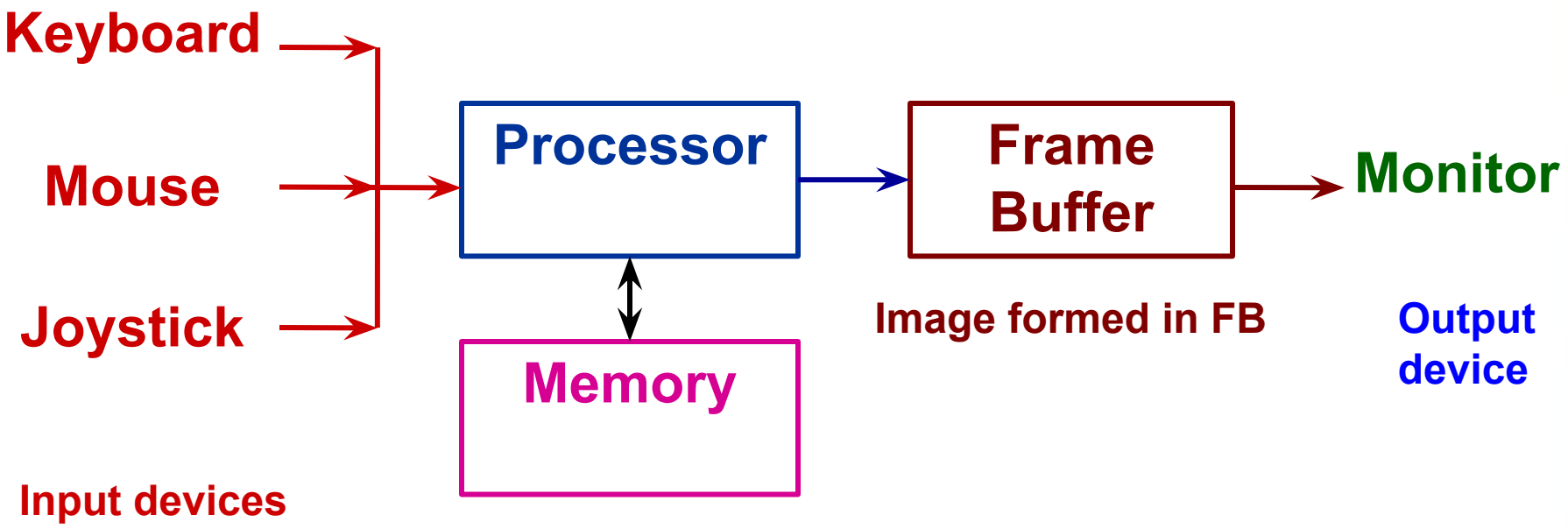


Input devices

Output device

Image formed in FB

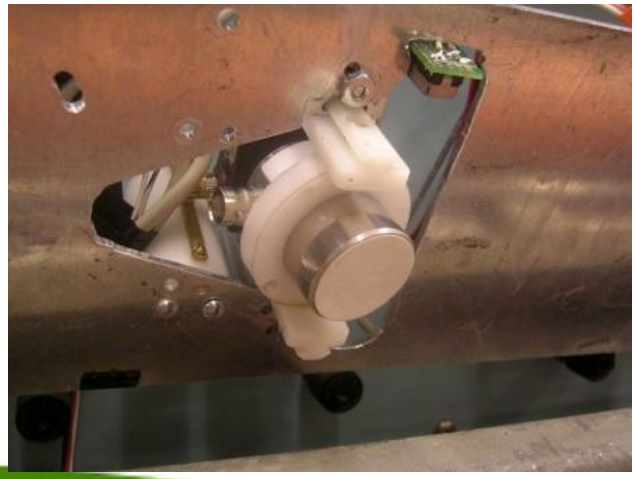
Basic Graphics System



Input Devices

Input Devices

- Keyboard + another input device
  - like **mouse, joystick or data tablet** - called as pointing devices.
  - Allow user to indicate a particular location on the display.
- Advanced Graphics systems allow 3D input devices
  - like **Laser range finders** *(uses a laser beam to determine the distance to an object)* and **acoustic sensors** *(to measure sound levels).*

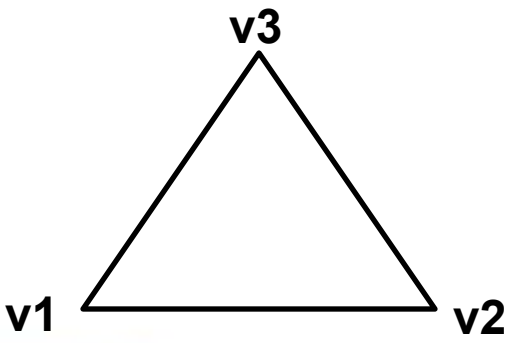


Processor

Processor

- **Graphical function of the processor**
  - to take specifications of graphical primitives (lines, polygons etc) generated by application programs.
  - to assign values to the pixels in the frame buffer.
- **Example :**

  A triangle is specified by its 3 vertices, but to display its outline by 3 line segments connecting vertices, graphics system must generate a set of pixels that appear as line segments to the viewer.



Processor

- All graphic systems have a **specialized graphics processor.**
- It is designed to carryout special graphics functions such as **rasterization.**

Rasterization (Scan Conversion)

- The process of converting primitives into pixel representation(2D image).
- Each point of this image contains information such as color and depth.

- Now a days, special purpose graphics processing units(GPUs) are used.
- GPU can be on mother board or on a graphics card.

Memory

Memory

Graphics system has two types of memory.

- General Memory
- Frame Buffer

General Memory – used to store ***non-graphics*** based data.

Frame Buffer – used to store ***graphics*** data.

Frame Buffer

- It is a special type of memory present on graphics system.
- In CG, a picture or an image is produced as an ***array of pixels(Raster).*** These pixels are stored in the frame buffer.
- ***Resolution of frame buffer***

  It is the no. of pixels in the frame buffer.

Frame buffer

- ***Depth (precision) of frame buffer***
  - No. of bits for each pixel is known as depth of frame buffer.
  - Determines properties like how many colors can be represented.

Ex:

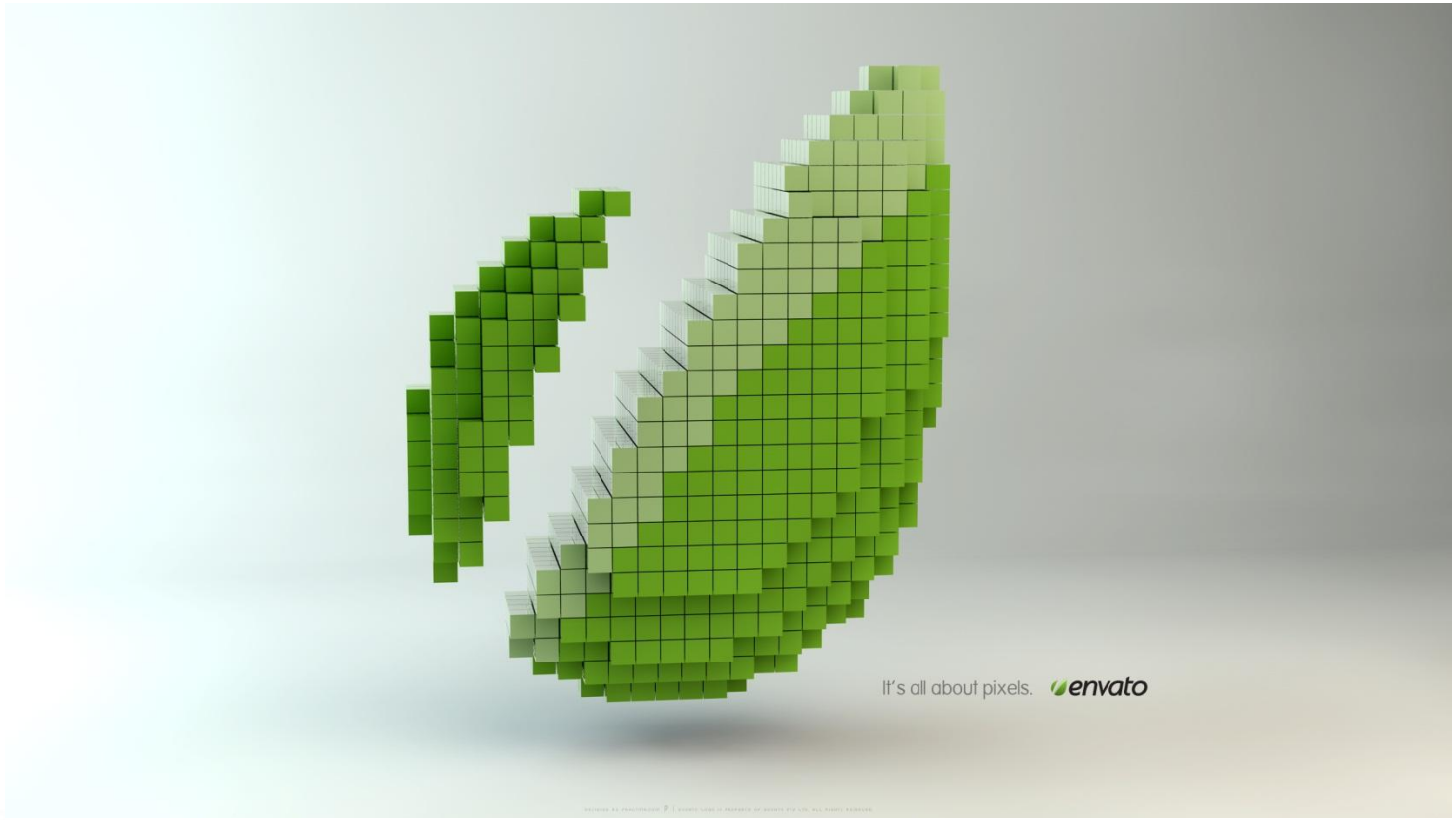
1-bit-deep frame buffer – 2 colors

8-bit frame buffer – 2^8 = 256 colors.

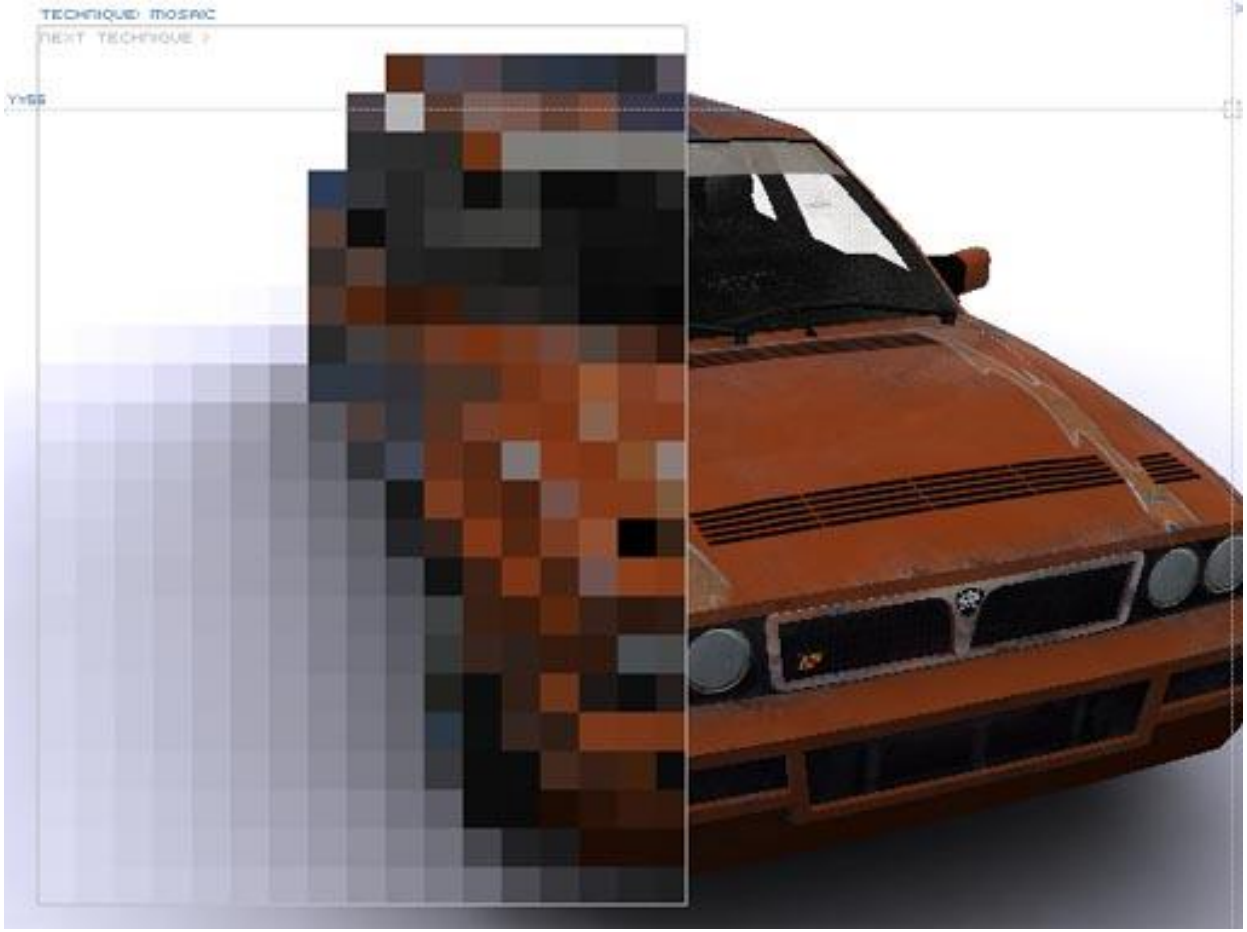
If depth=24 bits per pixel or more, such systems are known as Full color OR True color OR RGB color systems.

- In full color systems, there are 24(or more) bits per pixel. Can display sufficient colors to represent most images realistically.

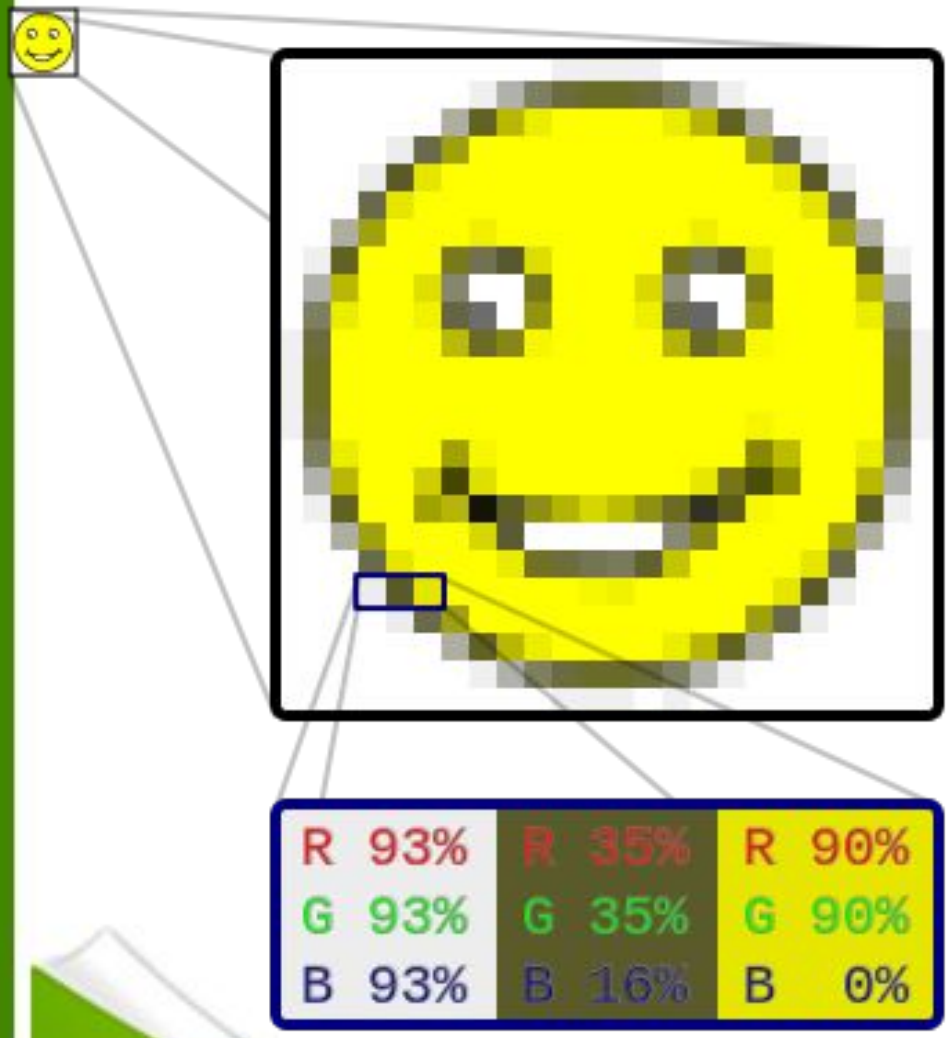
Pixels and Frame Buffer



Pixels



Pixels



The smiley face in the top left corner is a bitmap image. When enlarged, individual pixels appear as squares. Zooming in further, they can be analyzed, with their colors constructed by adding the values for red, green and blue.

Pixels

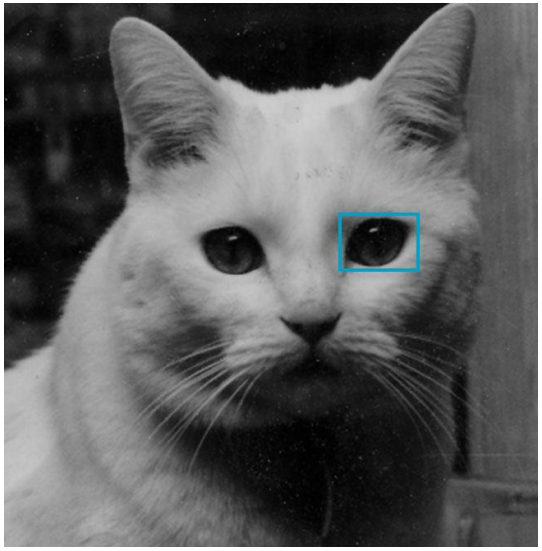
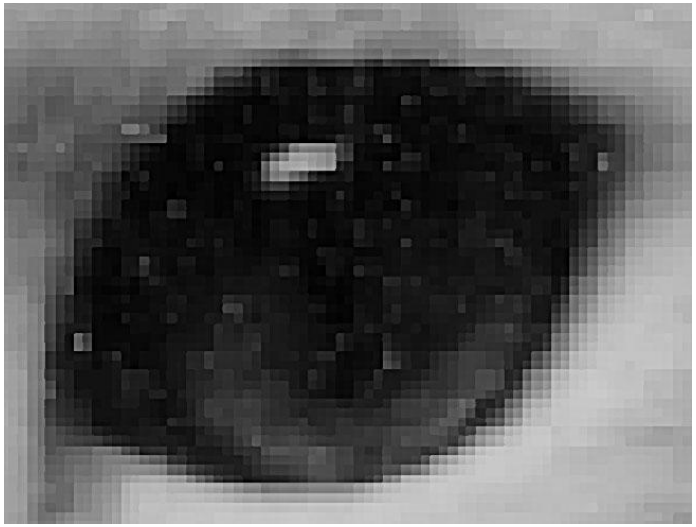


Image of a Cat using pixels



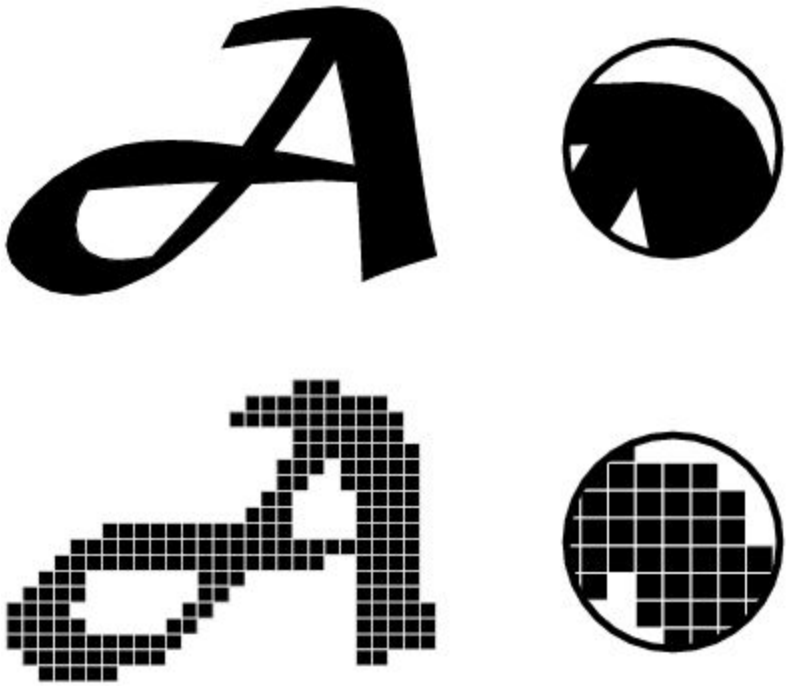
Detail of area around one eye showing individual pixels

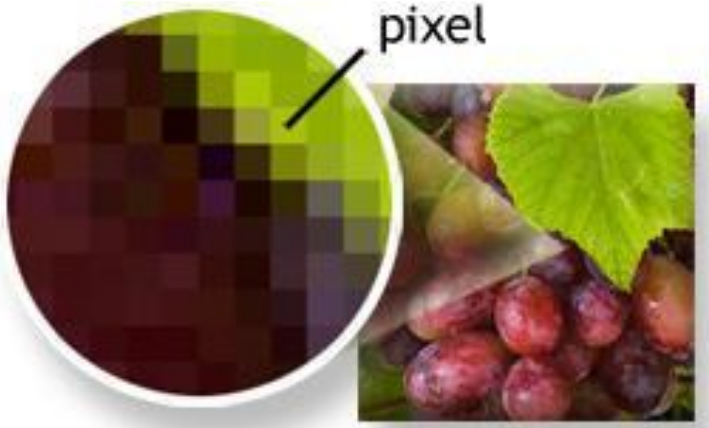
Pixels

High-resolution image, printed at 300ppi

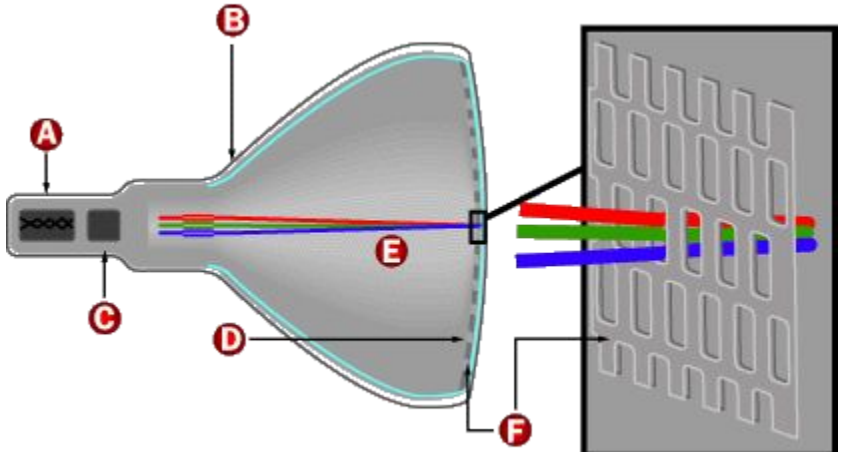
Low-resolution image, printed at 72 ppi

Pixels





Tiny Little Pixels

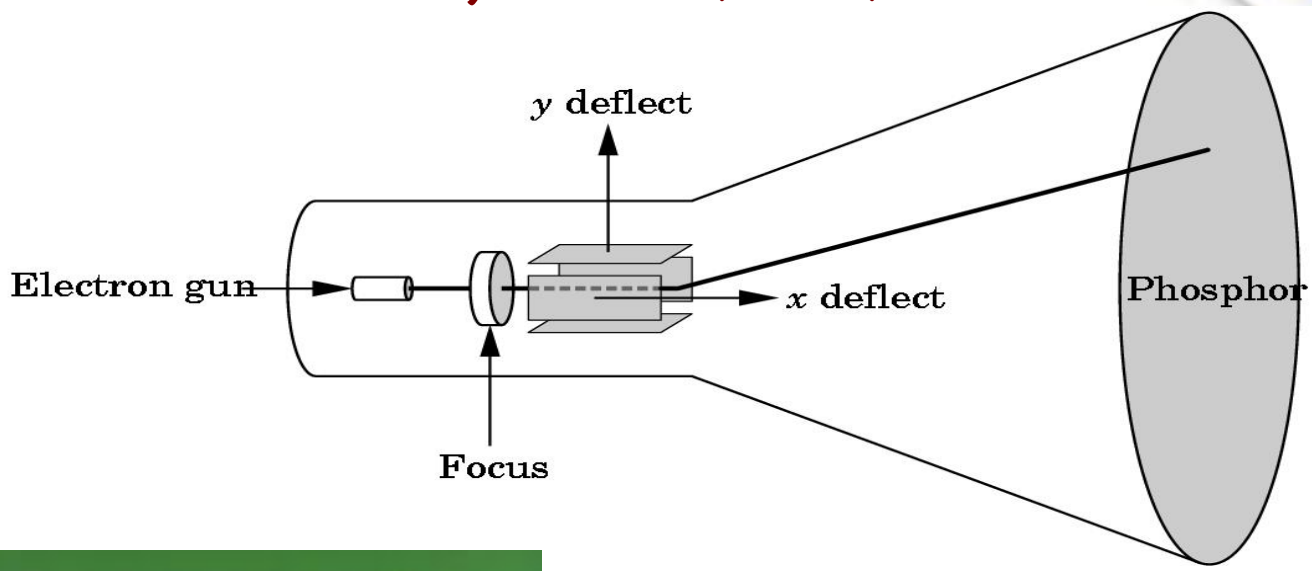


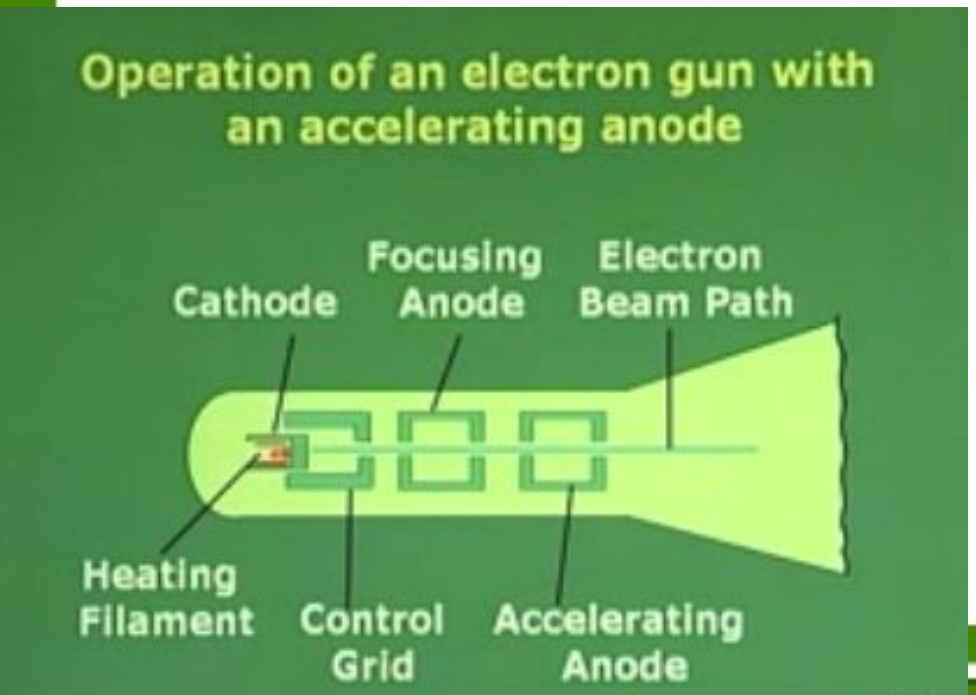
Output devices

Output devices

- For many years, dominant type of display is CRT (Cathode Ray Tube).
- They are rapidly being replaced by flat screen technologies such as LEDs, LCDs, plasma etc.

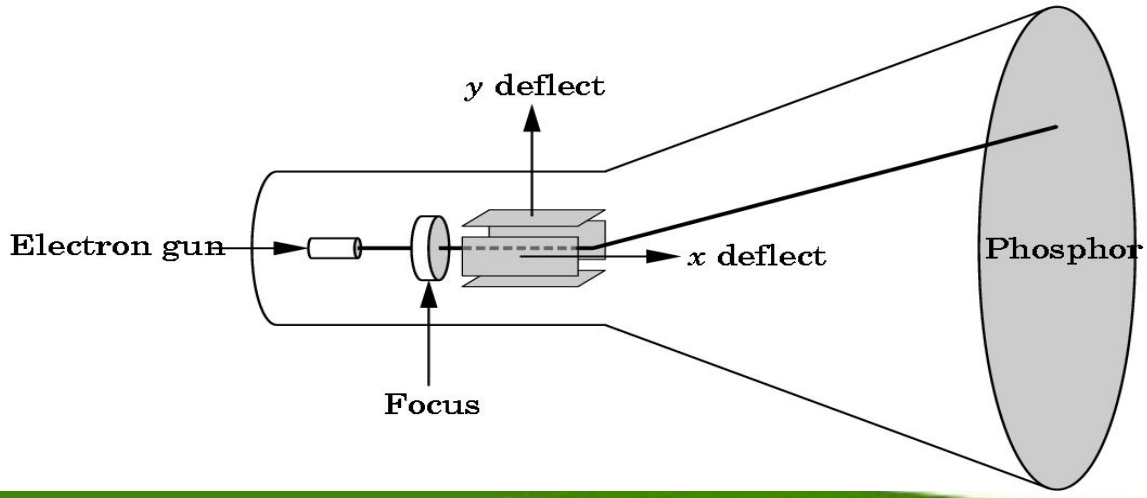
Working of Cathode Ray tube (CRT)





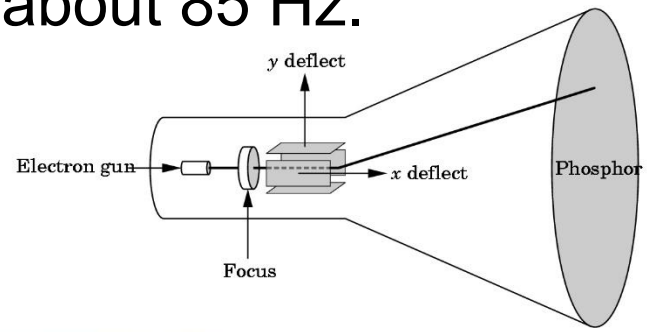
Working of Cathode Ray tube (CRT)

- The digital output produced by a processor is given to ***digital-to-analog converter.***
- The analog voltages produced are applied across the x and y deflection plates.
- The direction of the beam can be controlled by these two deflection plates.
- When electrons strike the phosphor coating on the tube, light is emitted.



Working of Cathode Ray tube (CRT)

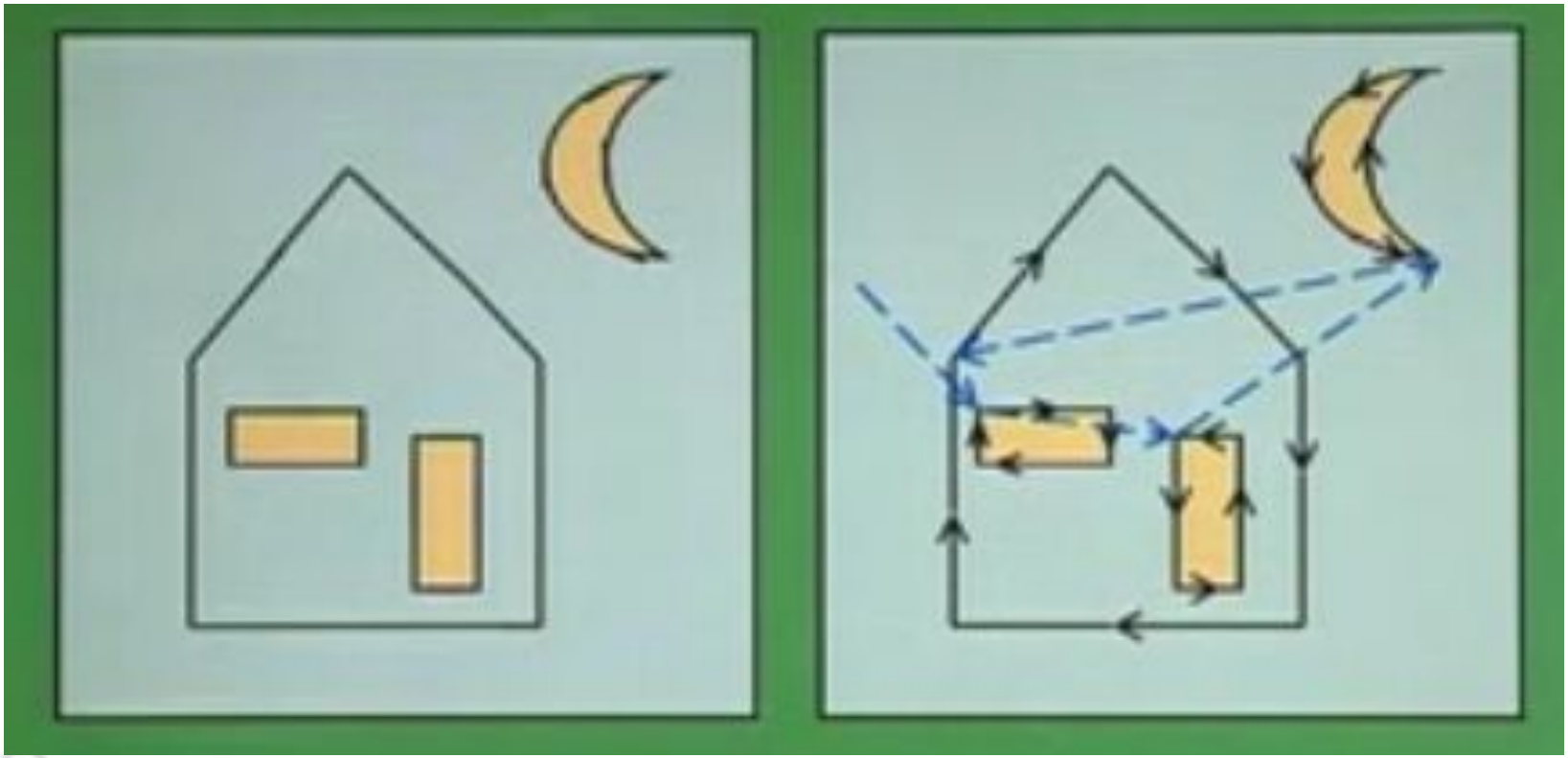
- Colored CRTs have 3 different colored phosphors (**RED**, **GREEN**, **BLUE**). They have 3 electron beams corresponding to 3 types of phosphors.
- A typical CRT will emit light only for a short duration when hit by an electron.
- This would produce flicker in the image.
- To produce a flicker-free image, the electron beam must be refreshed at a high rate called ***refresh rate.***
- Modern displays have refresh rate of about 85 Hz.



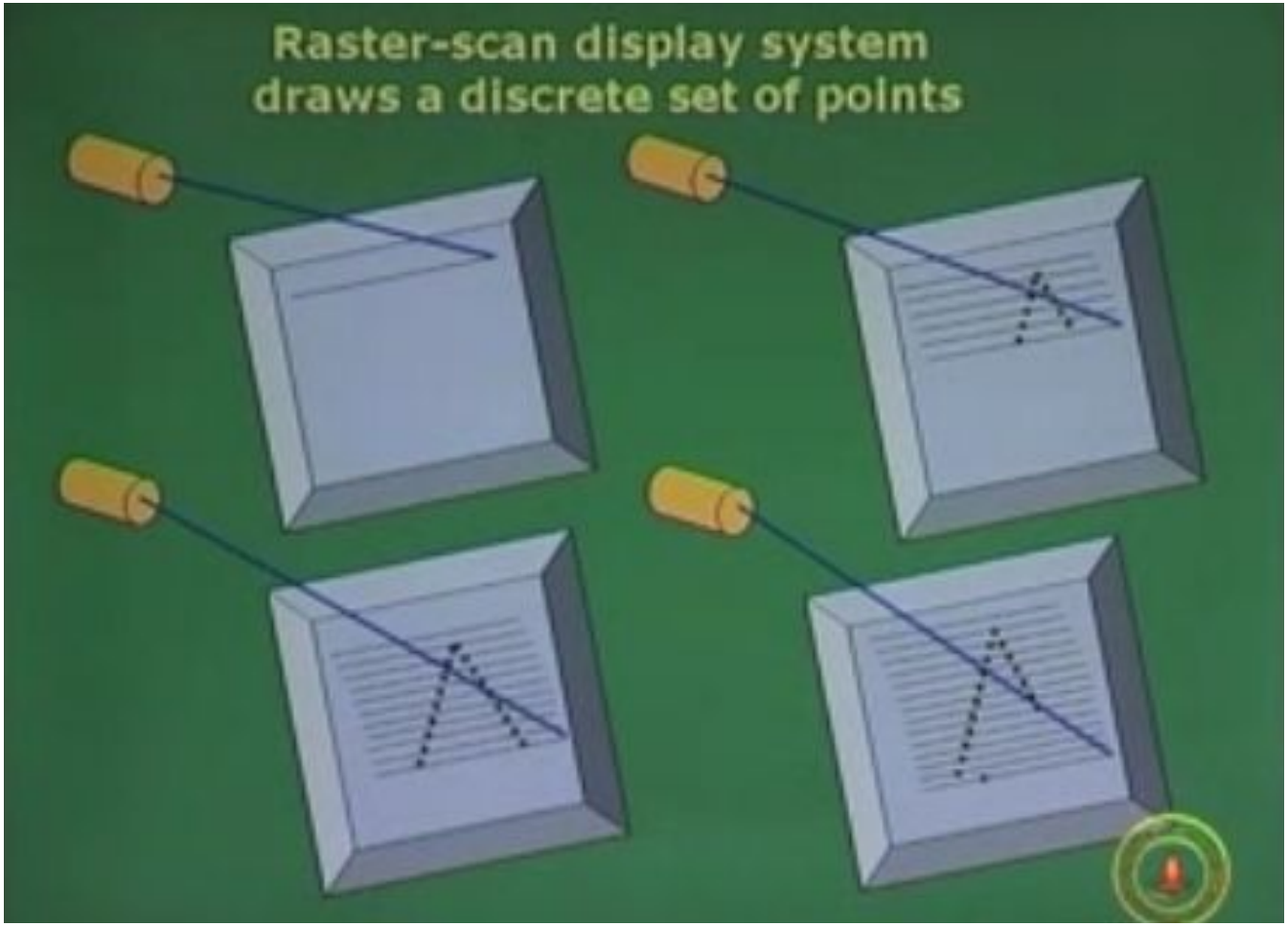
Types of CRTs

Random Scan CRT	Raster system CRT
Also called as calligraphic OR vector CRT. **Characters are made of sequence of strokes.**	**Entire screen is a matrix of pixels.** It takes pixels from frame buffer and displays them as points on the surface of the CRT. (Noninterlaced & interlaced displays)
A beam can be moved from any position to any other position.	Line cannot be drawn directly from one point to another.
Limited use.	Widely used.
Less efficient.	More efficient and can generate any image.

Random scan CRT

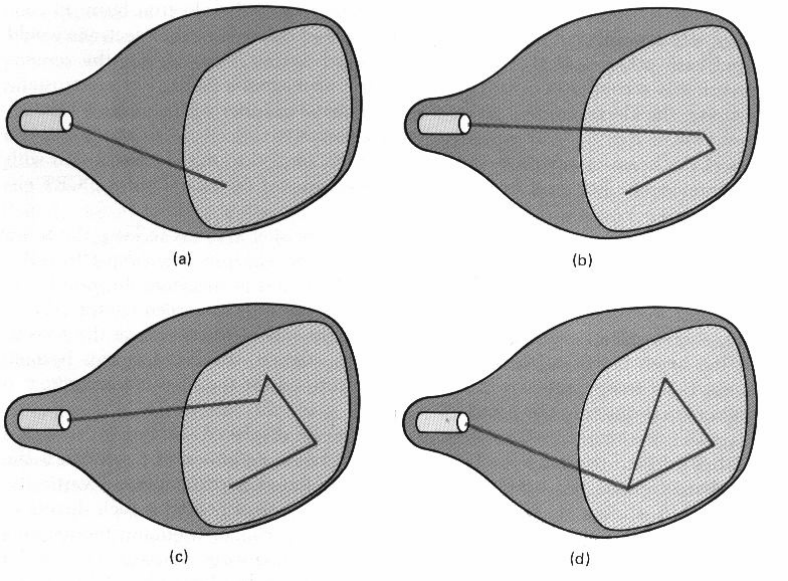


Raster System CRT

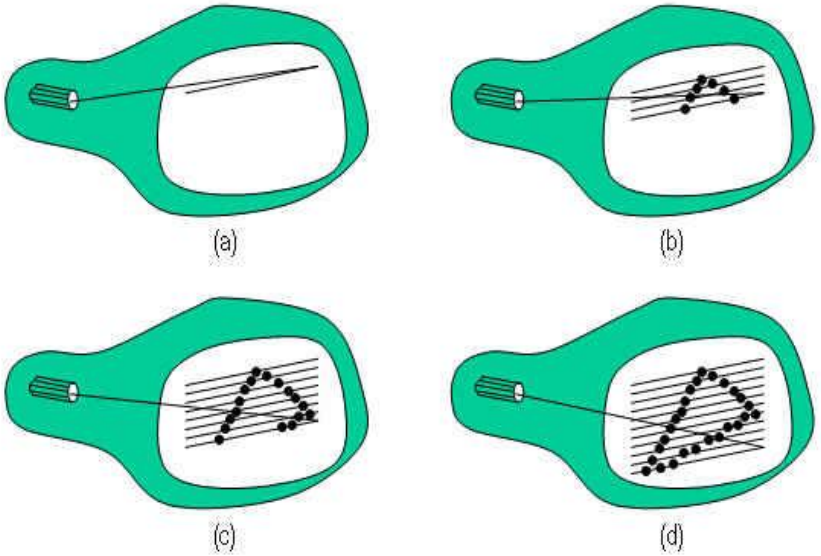


Types of CRTs

Random scan CRT



Raster system CRT

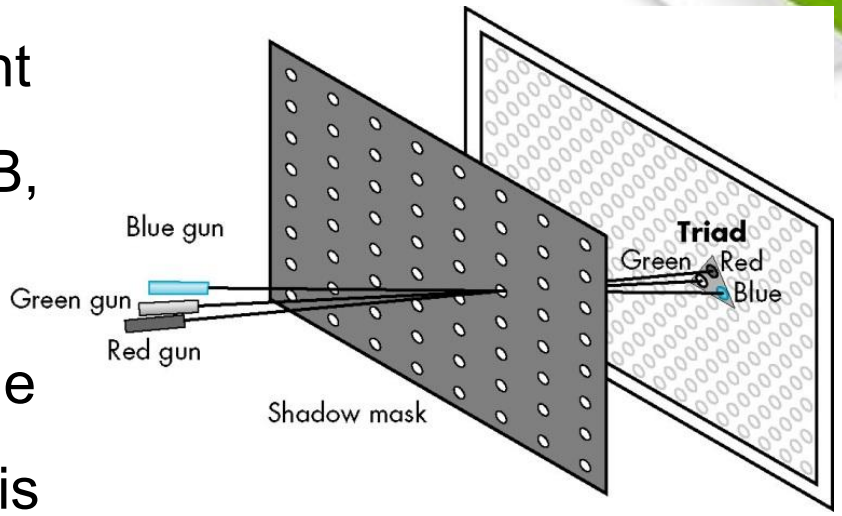


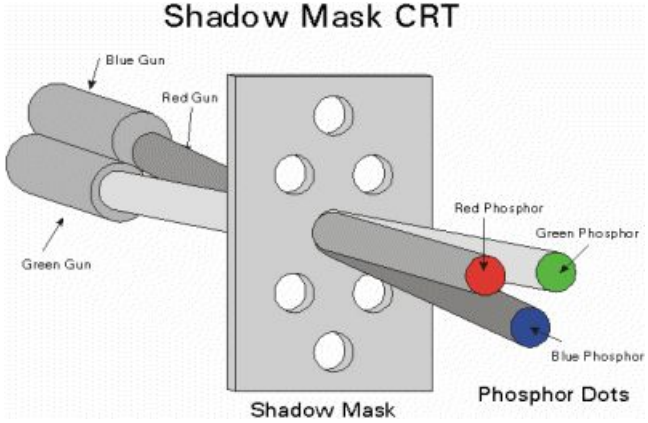
Shadow-mask CRT – colored CRT

The shadow mask is one of the technologies used to manufacture cathode ray tube (CRT) televisions and computer displays that produce color images.

Shadow-mask CRT – colored CRT

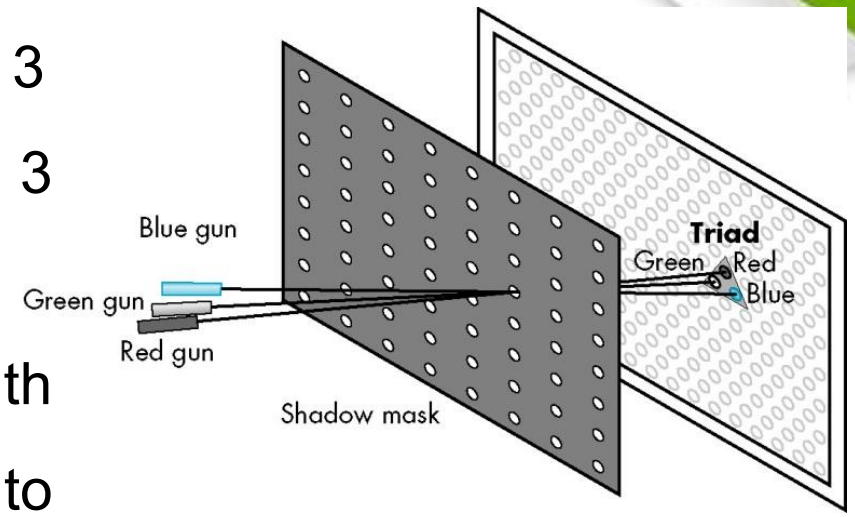
- Color CRTs have 3 different colored phosphors – RGB, arranged in small groups.
- A common style that arranges the phosphors in triangular groups is called as **Triads,** each triad consisting of 3 phosphors one for each primary.

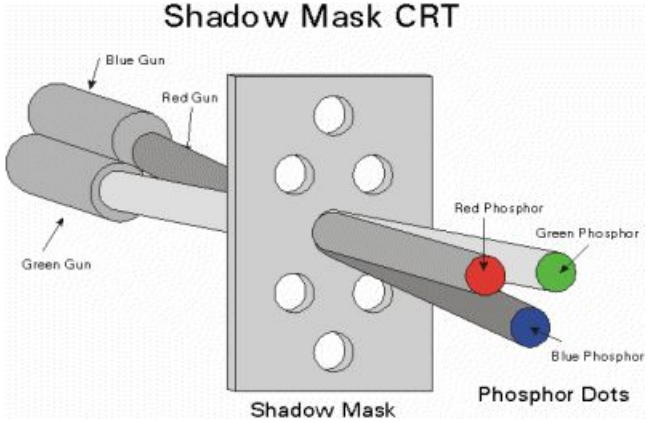




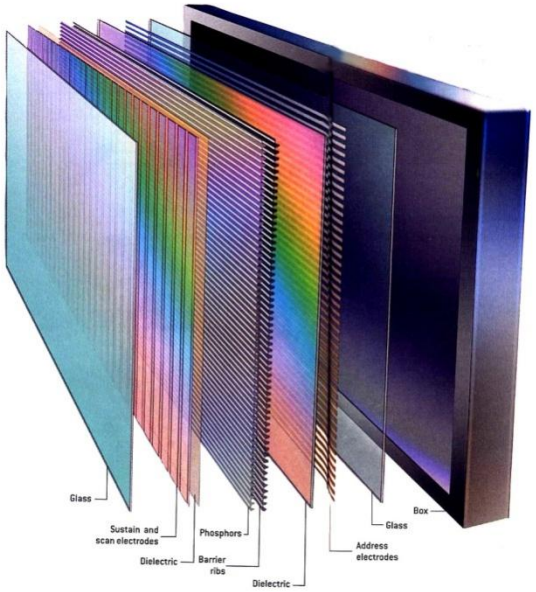
Shadow-mask CRT – colored CRT

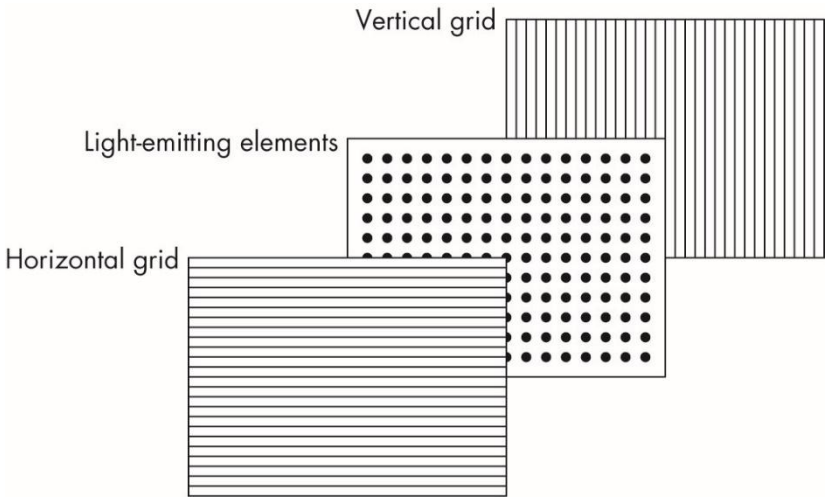
- Most of the color CRTs have 3 electron beams corresponding to 3 types of phosphors.
- Shadow mask, a metal screen with small holes(shadow mask), to ensure that an electron beam excites only phosphors of the proper color.





Generic flat-panel display



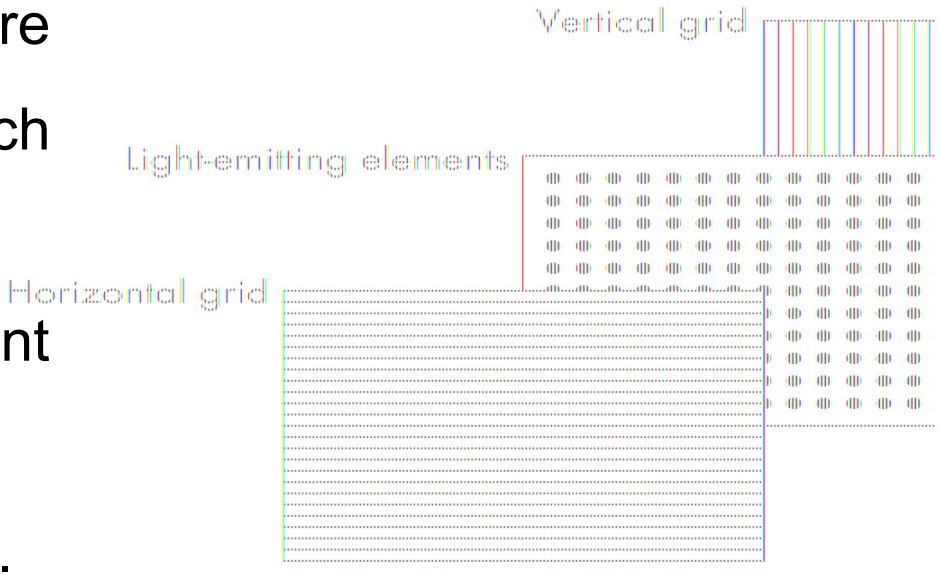


Popular displays

- LED
- LCD
- Plasma panels

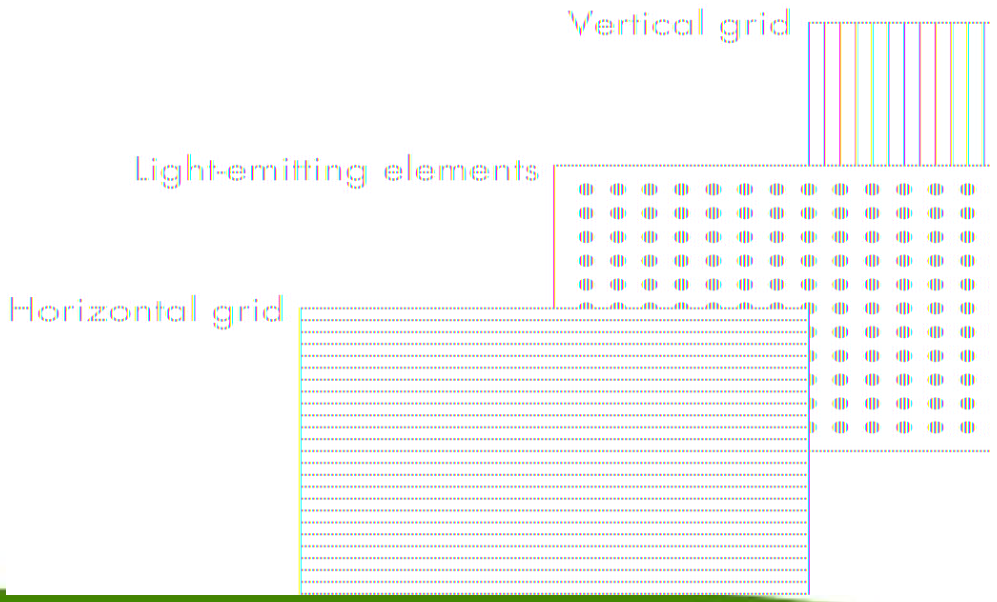
Generic flat-panel display

- The two outside plates contain parallel grids of wires that are oriented perpendicular to each other.
- The middle panel is different for the 3 types of displays.
- LED, LCD and Plasma panel.



Generic flat-panel display

- By sending electric signals to the proper wire in each grid, the electric field at a location can be made strong enough to control the corresponding element in the middle plate.
- Middle plate can be LED panel or LCD panel or plasma panel.



Generic flat-panel display

LED display

The middle panel contains Light emitting Diodes which are turned ON and OFF by the electrical signals sent to the grid.

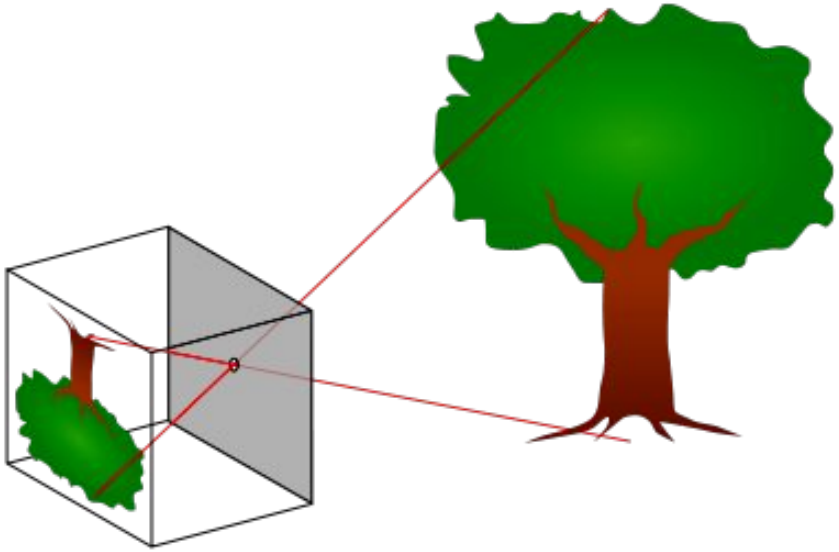
LCD display

Electrical field controls the polarization of liquid crystals in the middle panel.

Plasma

Uses voltage on the grids to energize gases embedded between the glass panels holding the grids. The energized gas becomes glowing plasma.

Objects and viewers



Objects and viewers

Objects are created using Primitives

Images are created from object using Camera

Objects and viewers

Basic entities that are part of any image formation process

1. Objects
2. Viewers

Objects

- Physical structure of the image.
- In graphics, we form objects by specifying the positions in space of various geometric primitives (lines, points, polygons).

Objects

A sample OpenGL code that specifies triangular polygon.

```
glBegin(GL_POLYGON);
        glVertex3f(0.0,0.0,0.0);
        glVertex3f(0.0,1.0,0.0);
        glVertex3f(1.0,0.0,0.0);
glEnd();
```

Viewer

- One who forms image of the object.
- Objects exist in real world and it is 3D. A viewer sees an object as image in 2D.
- Viewer can be Human, camera or digitizer.

Image formation

It is the process by which the specification of the object is combined with specification of the viewer to produce 2D image.

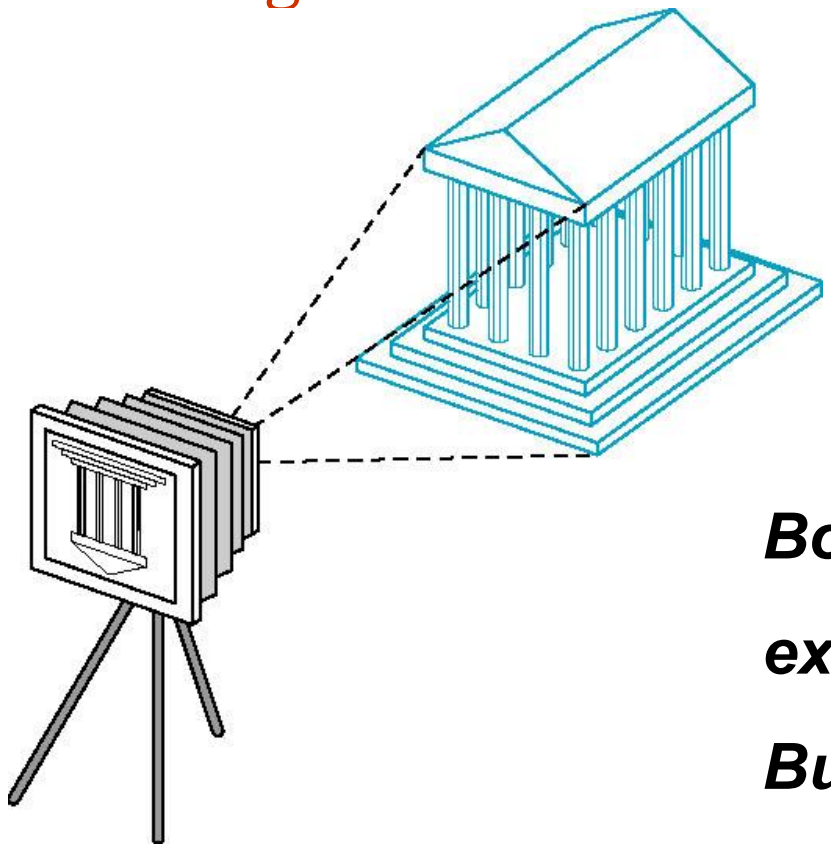
Imaging system

- **Any object is always 3 dimensional. Image is always 2 dimensional.**
- So an object from 3D is converted into image in 2D representation.
- Elements of Image formation
  - Object
  - Camera (viewer)
  - Light source

Imaging system

- Every imaging system must provide means of forming images from objects.
- To form an image, we must have someone viewing our objects (person, camera etc).
- **It is the viewer that forms the image of the objects.**
- In human visual system, image is formed at the back of the eye and in a camera, on a film plane.

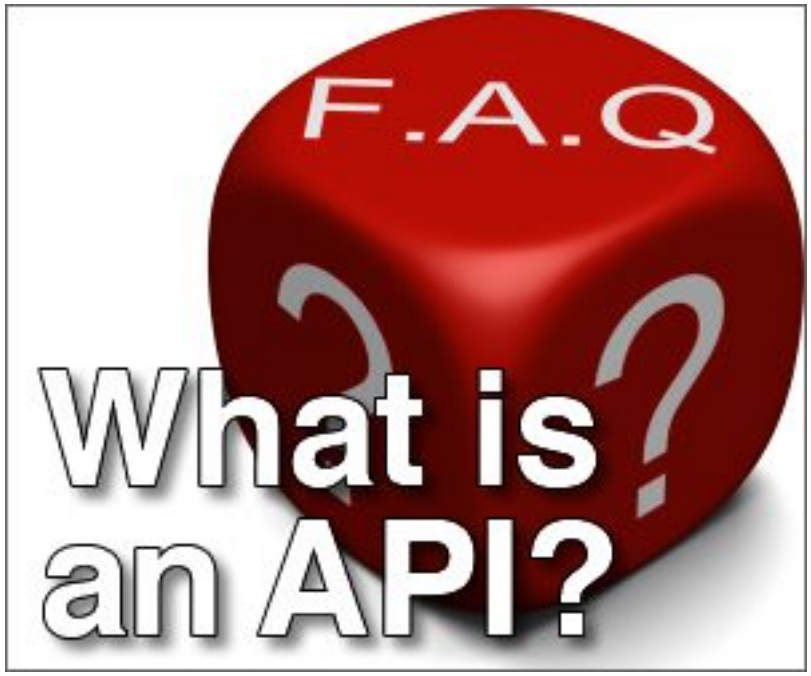
A camera system viewing image of a building



Both object and viewer exist in 3-D world.

But the image that they define, on film plane, is 2-D.

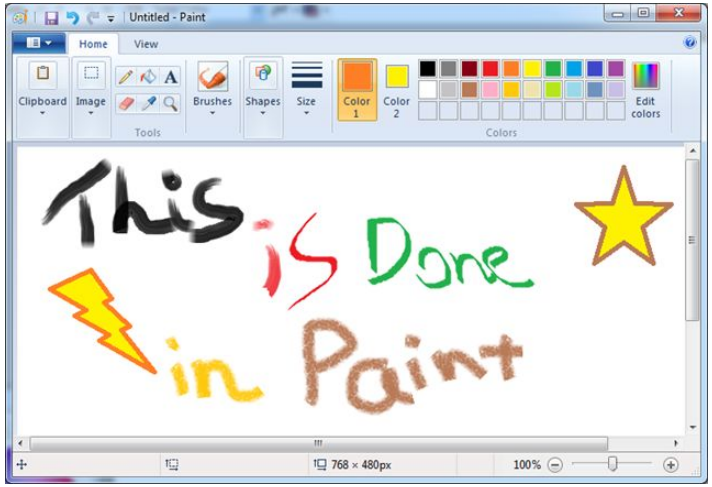
Application Programmer Interface



The Programmer's Interface (PI / API)

Interaction in paint program

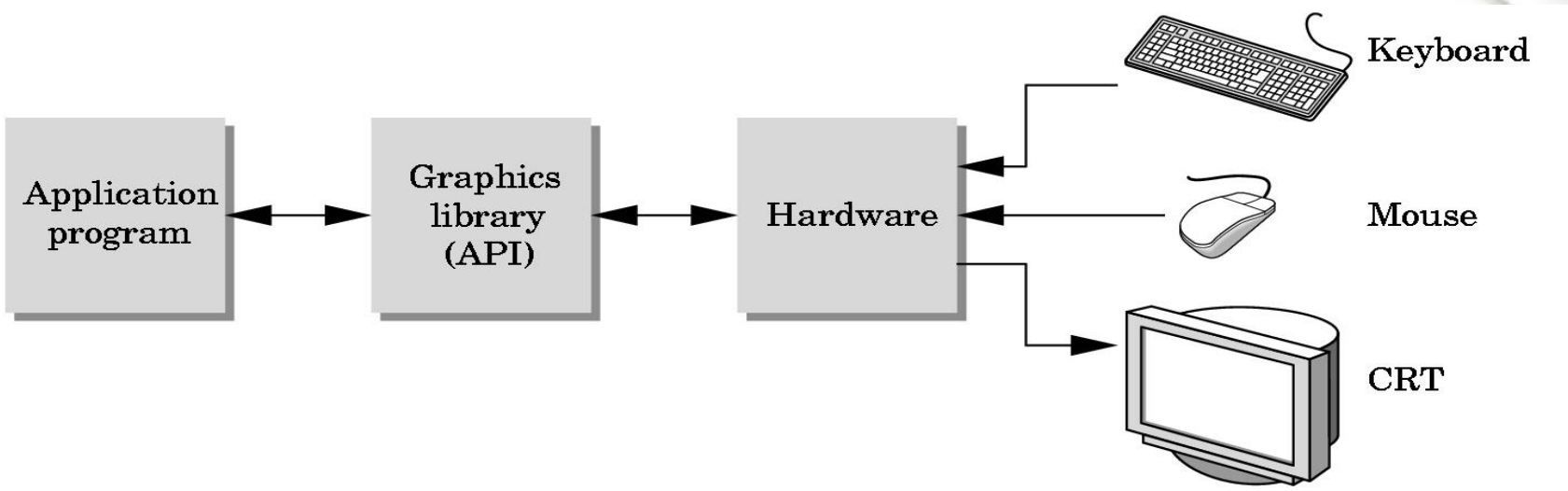
- User develops images through interactions with the menus and icons using input devices.
- By clicking the menus or icons, the user guides the software and produces images without having to write programs.



The Programmer's Interface (PI / API)

- The **interface** between an application program and a graphics system can be specified through a set of functions that resides in a graphics library.
- These specifications are called the **application programmer's interface.**

The programmer's interface



- The application programmer writes an application program using graphics functions supported by the API.
- The output of the API is given to the software drivers which convert the data to a form understood by the particular hardware.

Three-Dimensional APIs

- ***Synthetic camera model*** is the basis for all popular APIs including openGL, Direct3D and Open Scene Graph.
- This model gives APIs to specify the following.
  - Object
  - A viewer
  - Light sources
  - Material properties

Objects in OpenGL

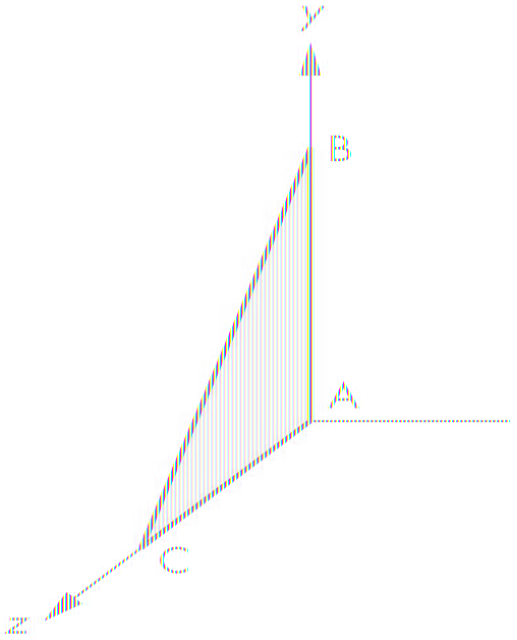
- Defined by ***"set of vertices".***
- Example : A rectangle can be created in OpenGL using 4 vertices.
- In graphics, we form objects by specifying the positions in space of various geometric primitives (lines, points, polygons).

Objects in OpenGL

OpenGL programs define primitives through **list of vertices.**

Ex., triangular polygon

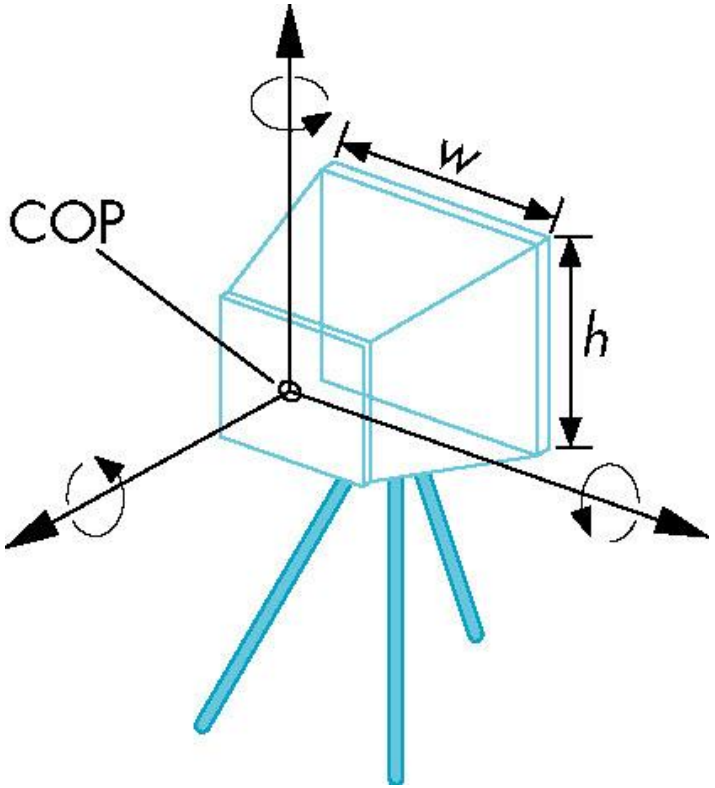
```
glBegin(GL_POLYGON);
    glVertex3f(0.0, 0.0, 0.0); /* vertex A*/
    glVertex3f(0.0, 1.0, 0.0); /* vertex B*/
    glVertex3f(0.0, 0.0, 1.0); /* vertex C*/
glEnd();
```



Viewer or camera

A viewer can be specified in a variety of ways. The 4 types of specifications are

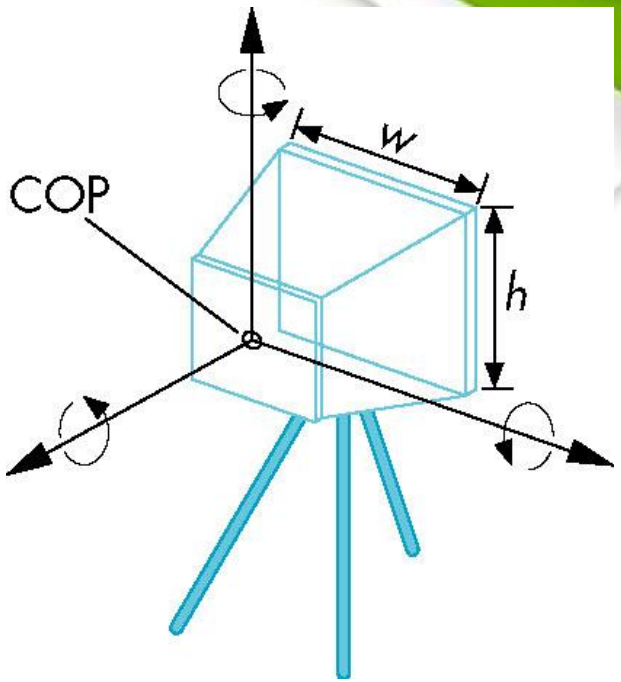
1) **position**
2) **Orientation (rotation)**
3) **Focal length**
4) **Film plane**



Viewer or camera

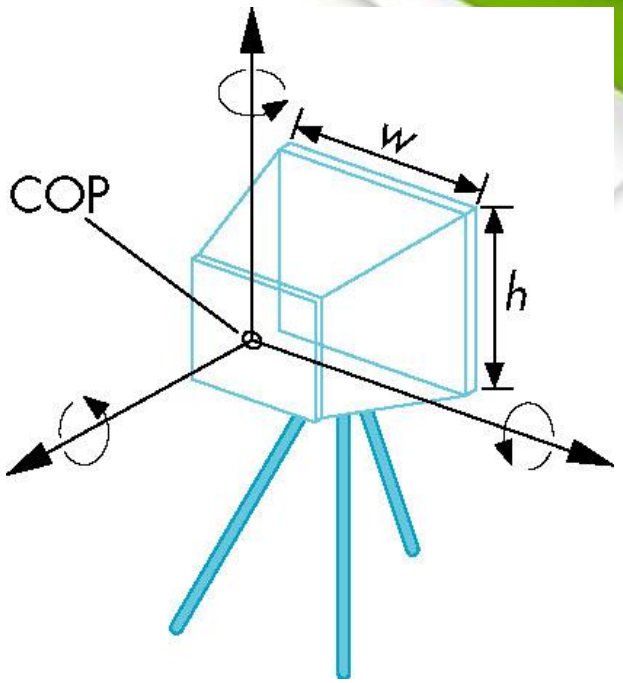
1. **Position:** The camera location usually is given by the position of the center of the lens (**the center of projection**).

2. **Orientation:** Once the camera is positioned, a camera coordinate system can be placed with its origin at the center of projection. Then the camera can be rotated independently around the three axes of this system.



Viewer or camera

3. **Focal length:** The focal length of the lens determines the **size of the image** on the film plane or, equivalently, the portion of the world the camera sees.



4. **Film plane:** The back of the camera has a height and a width. The orientation of the back of the camera can be adjusted independently of the orientation of the lens.

Viewer Specification in OpenGL

1. gluLookAt()
2. gluPerspective()

1. gluLookAt(cop_x, cop_y,cop_z, at_x, at_y, at_z, up_x, up_y, up_z);

- points the camera from COP toward desired point with specified up direction for the camera
- cop_x,cop_y, cop_z : eyePosition3D is a XYZ position. This is where you are (your eye is).
- at_x, at_y, at_z : center3D is the XYZ position where you want to look at.
- up_x, up_y, up_z : upVector3D is a XYZ normalized vector. Often 0.0, 1.0, 0.0

Viewer Specification in OpenGL

void **gluLookAt (**

GLdouble *eyex*, GLdouble *eyey*, GLdouble *eyez*,

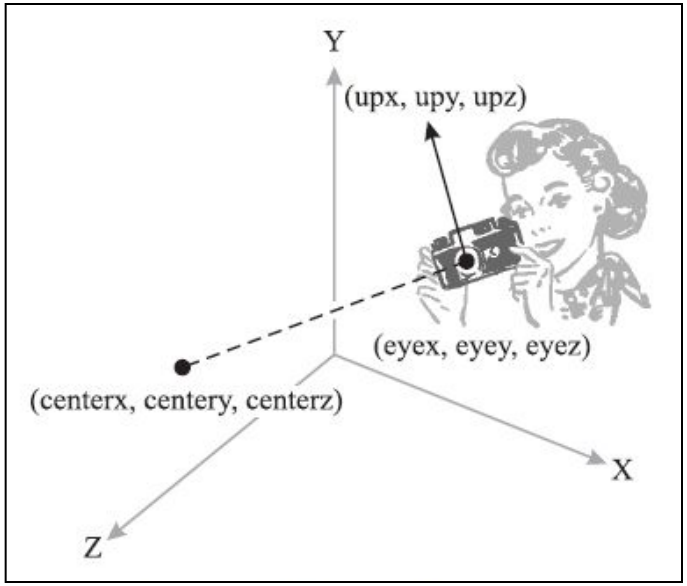
GLdouble *centerx*, GLdouble *centery*, GLdouble *centerz*,

GLdouble *upx*, GLdouble *upy*, GLdouble *upz*

);

If you don't call **gluLookAt(), the OpenGL camera is given some default settings:**

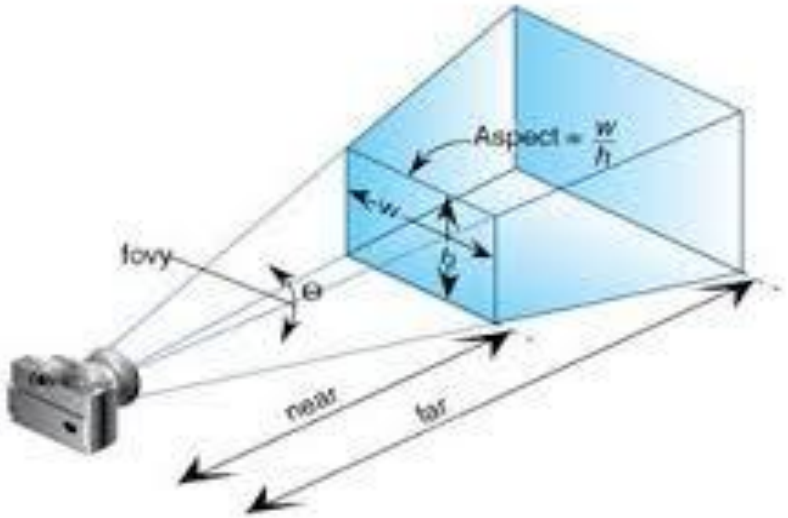
- it's located at the origin, (0, 0, 0);
- it looks down the negative Z axis;
- its "up" direction is parallel to the Y axis.



Viewer Specification in OpenGL

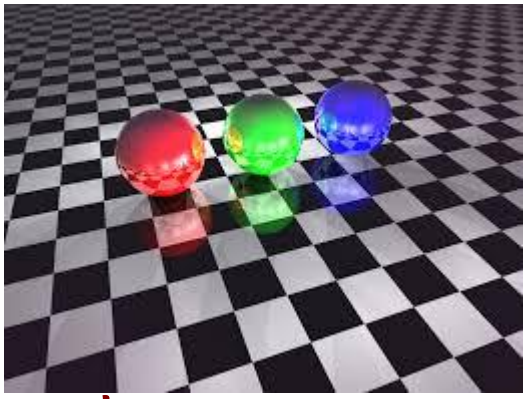
gluPerspective(field_of_view, aspect ratio, near, far);

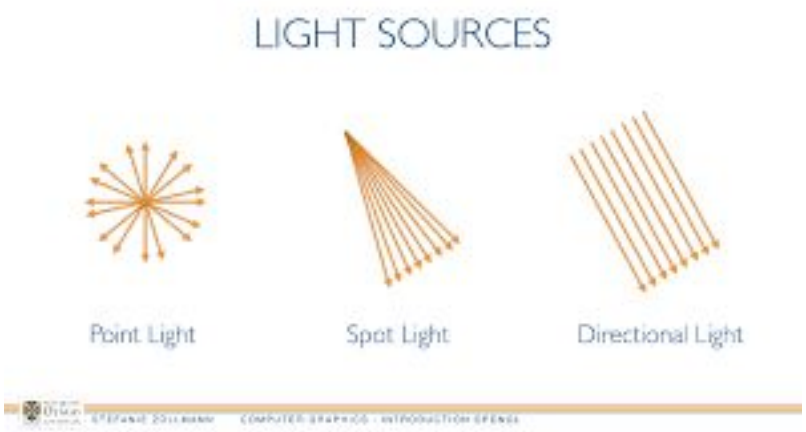
selects a lens for a perspective view and how much of the world the camera should image.



Light sources

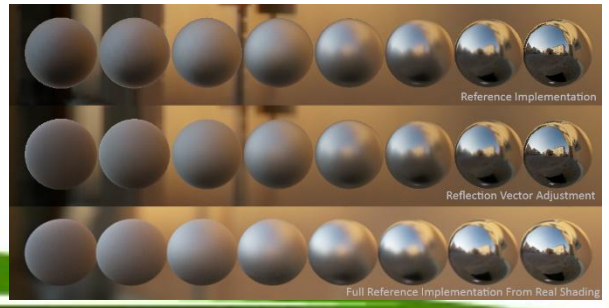
- Defined by their **location, strength, color and direction.**
- APIs provide a set of functions to specify these parameters.





Material Properties

- Material properties are ***attributes*** of the objects.
- Such properties are usually specified through a series of function calls at the time that each object is defined.



Graphics Architectures

Graphics Architectures

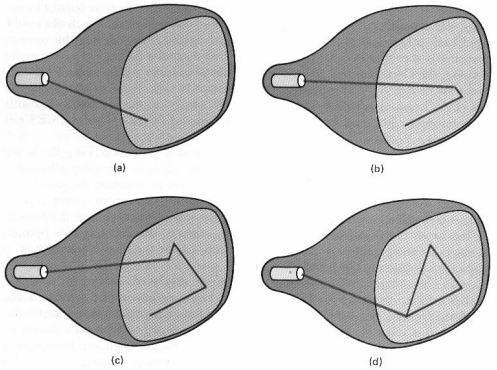
Graphics architectures have evolved over decades to reach the current stage.

Categories of Graphics architecture

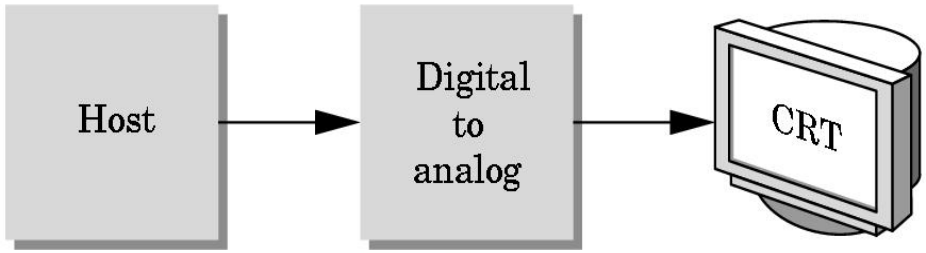
- Early Graphics Architectures (EGA)
- Display Processor Architectures (DPA)
- Pipeline Architectures

1. Early Graphics Architecture

- Used computers with standard Von Newmann architecture.
- Had **single processing unit** and could process **only one instruction at a time.**
- **Calligraphic CRT** display was used.



- ***Job of host***
  - To run application program.
  - compute end points of line segments in image.
  - To send these points to the display unit at a rate high enough the avoid flicker.



Contd...

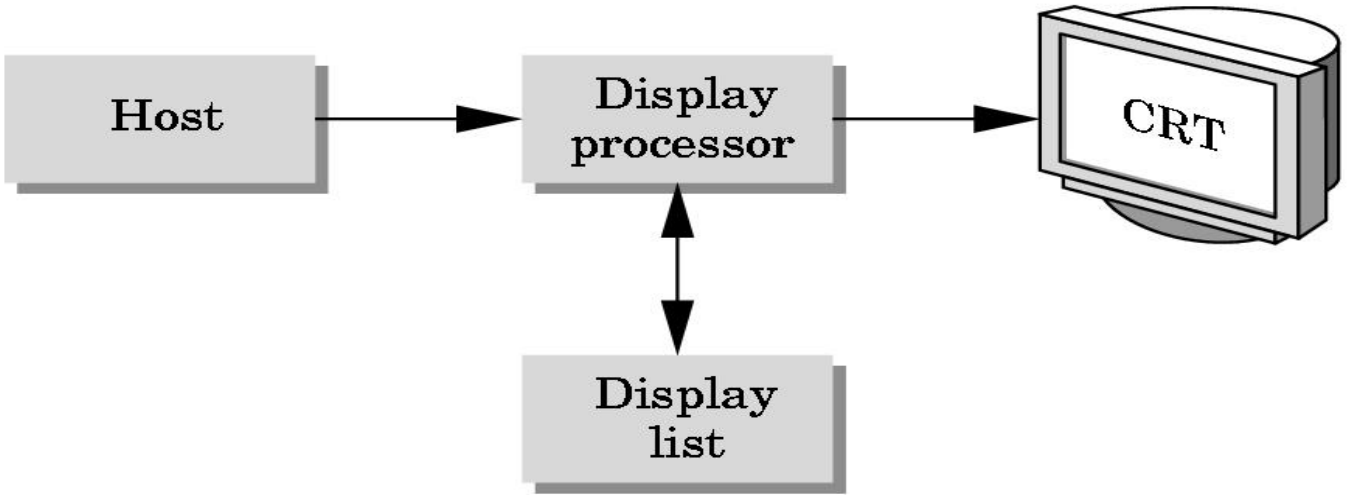
Advantages

- Simple architecture
- Very less expensive

Disadvantages

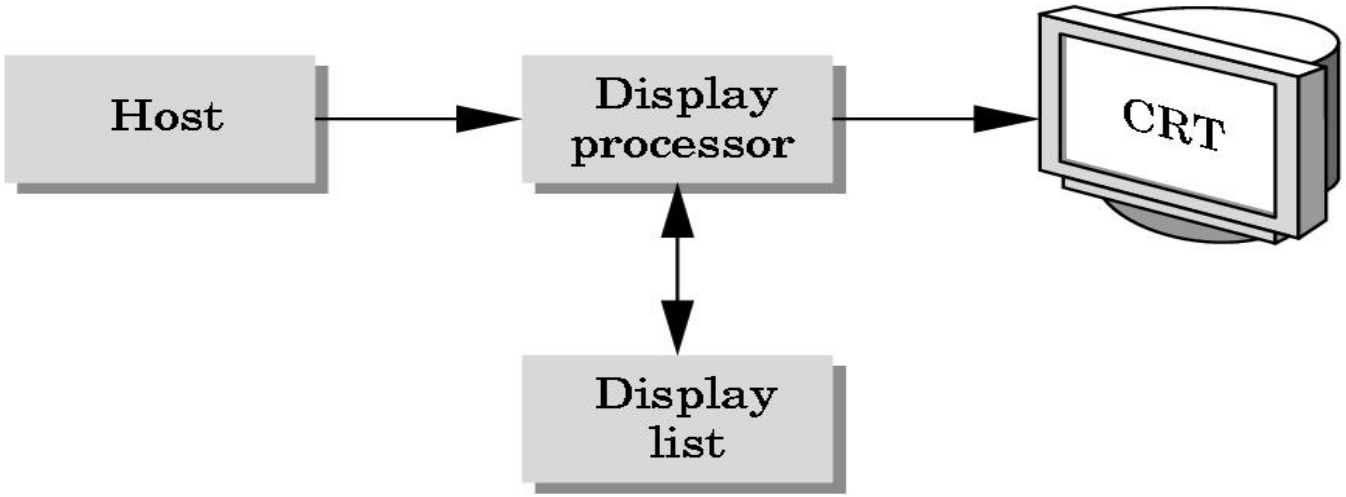
- Very slow
- Not suitable to produce all types of images. Hence EGA were replaced by DPA.

2. Display Processor Architecture



- It had the host computer , display processor, display list(display file) and the output device.
- The host computer would assemble the instructions (which can generate the image) once and sends it to the display processor.

2. Display Processor Architecture



- The display processor would store these assembled instructions in its own memory called display list.
- The display processor would then execute the display list continuously ***independently of the host*** at a rate which is sufficient to avoid flicker.

2. Display Processor Architecture

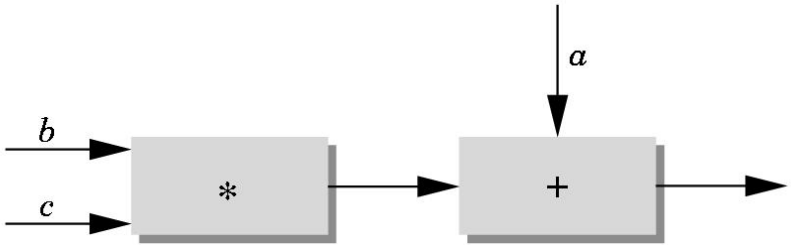
Advantages

- It frees the host computer from repeatedly assembling the instructions.
- Hence the host is free to perform other tasks.

Disadvantages

- Not suitable to produce real time 3D graphics images. Hence it was replaced by pipeline architecture.

3. Pipeline Architecture



- Adder and multiplier.
- *To calculate a + (b * c)*
- With many values of a,b,c: multiplier can pass on the results of its calculations to the adder and can start its next multiplication while adder carries out second step of calculation on the first set of data.
- So, the rate at which data flows through the system, the **throughput of the system is doubled.**

3. Pipeline Architecture

- Currently used and the most prominent graphics architecture.
- Makes use of **VLSI circuits** to generate images.
- The major steps involved to form an image
  - **Vertex processing**
  - **Clipping and primitive assembly**
  - **Rasterization**
  - **Fragment processing**

3. Pipeline Architecture

a) Vertex processing

- First step of Pipeline Architecture.
- **Each vertex is processed independently.**
- Carries out 2 functions
  - ***co-ordinate transformation***
  - ***computing color of each vertex***
- Successive change of co-ordinate system is represented as a single matrix.

3. Pipeline Architecture

a) Vertex processing

- Transformation can be done by multiplying OR concatenating individual matrices to a single matrix.
- After multiple stages of transformation, the geometry is transformed by a projection matrix.
- It also specifies color, properties of the object and light source in the scene.
- The result of this stage is a **set of vertices** that specify the object or group of objects.

3. Pipeline Architecture

b) Clipping and Primitive Assembly

- Second step of Pipeline Architecture.
- Output of vertex processing is given as input to this stage.
- This stage is manly involved in clipping of the primitive.
- ***Clipping***

  Just as a real camera cannot “see” the whole world, the virtual camera can only see part of the world or object space.

  Objects that are not within this volume are said to be clipped out of the scene.

3. Pipeline Architecture

b) Clipping and Primitive Assembly

Clipping

- Within this stage, we must assemble sets of vertices into primitives(line segments or polygons) before clipping takes place.
- The output of this stage is a **set of primitives** whose projections can appear in the image.

3. Pipeline Architecture

c) Rasterization

- Third step of Pipeline Architecture.
- Converts 2D objects into pixels.
- The primitives that are output by the clipper are processed to generate pixels in the frame buffer.
- The output of the rasterizer is a **set of fragments** for each primitive.
- Fragments are “potential pixels” having a location in frame buffer, have Color and depth attributes.

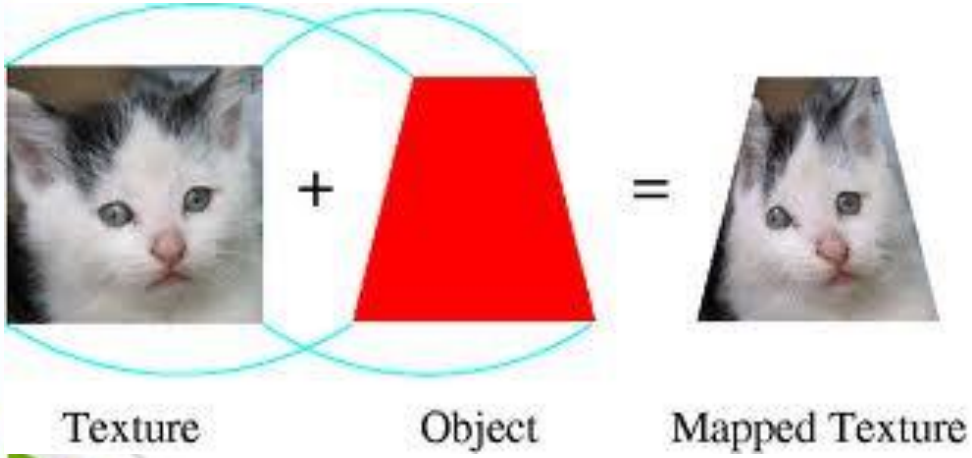
3. Pipeline Architecture

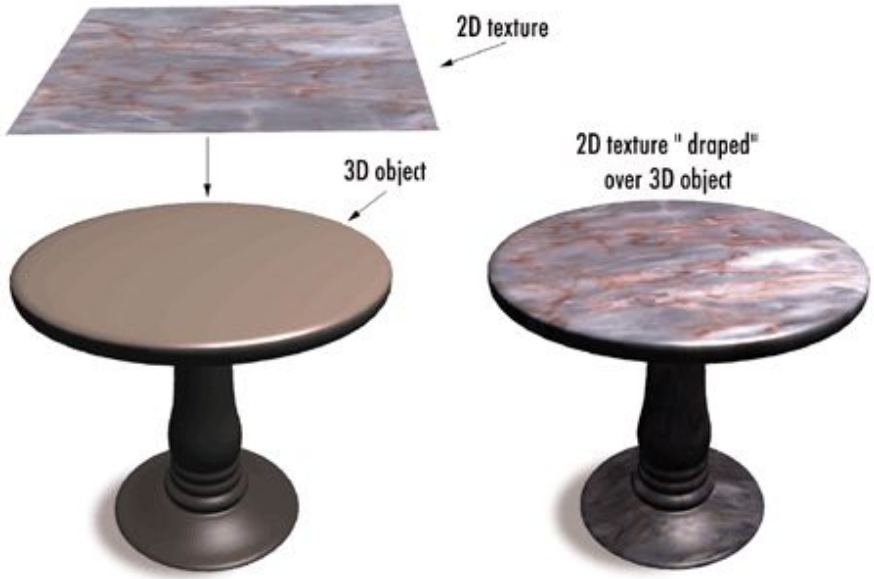
d) Fragment processing

- Final step of Pipeline Architecture.
- Fragments are processed to determine **the color of the corresponding pixel** in the frame buffer.
- Color of a fragment can be altered by **texture mapping.**

Contd..

Texture mapping is a method for adding surface texture (a bitmap or raster image), or color to a computer -generated graphic or 3D model. A **texture map** is applied (mapped) to the surface of a shape or polygon.

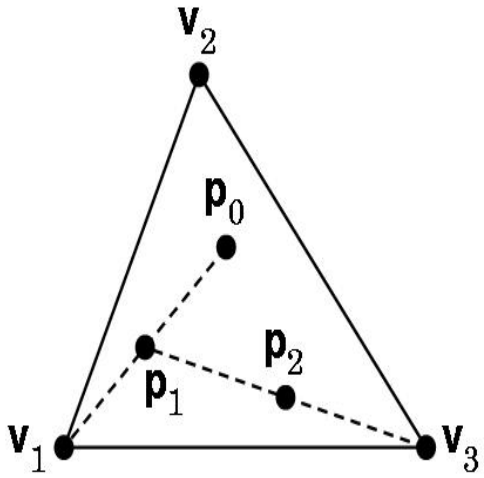




Construction of Sierpinski Gasket - using random numbers

The construction of sierpinski gasket as follows:

1. Pick up a random point inside the triangle.
2. Select one of the three vertices at random.
3. Find the point halfway between initial point and the randomly selected vertex.
4. **Display this new point** by putting some sort of marker, such as a small circle, at its location.
5. Replace the initial point with this new point.
6. Return to step 2.

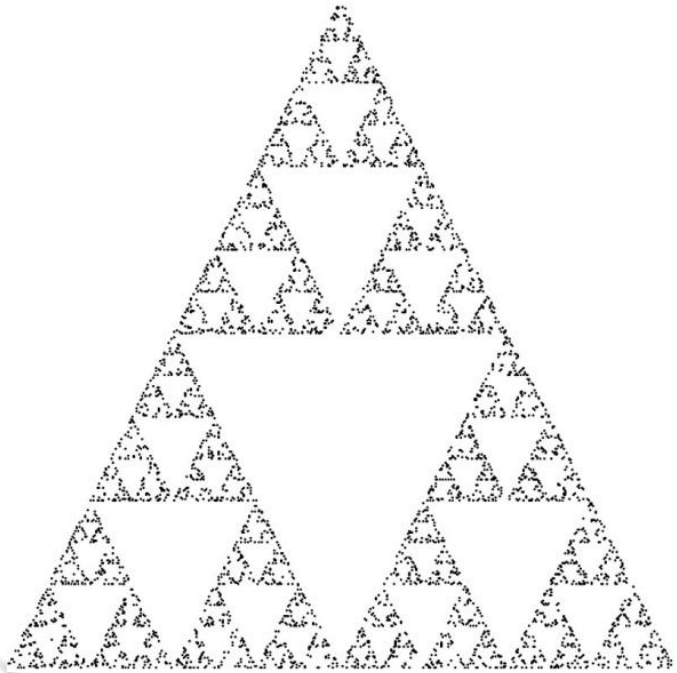


Thus, each time a point that is generated, it is displayed on the output device.

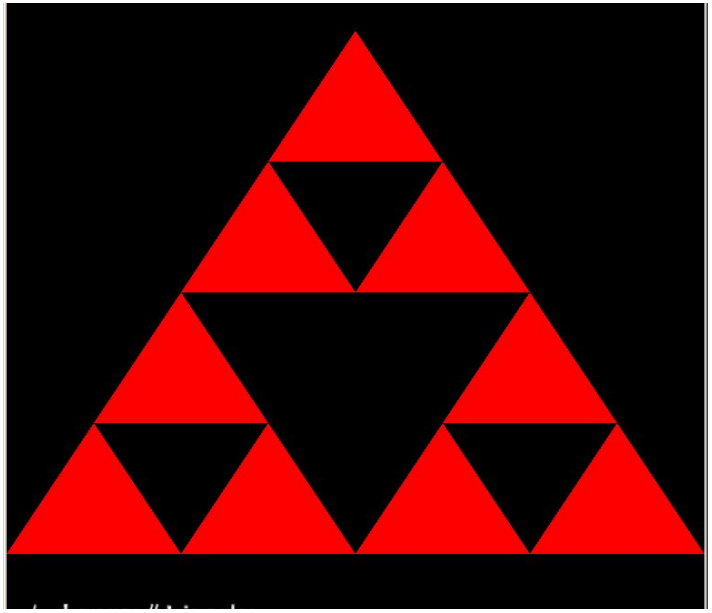
In the figure p_0 is the initial point, and p_1 and p_2 are the first two points generated by the algorithm.

Sierpinski Gasket Construction

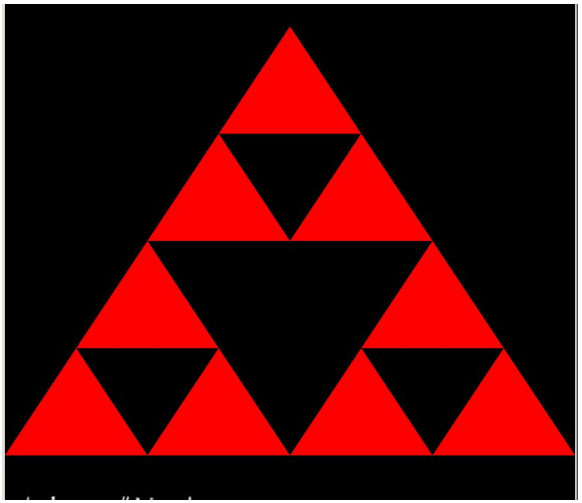
random number generation

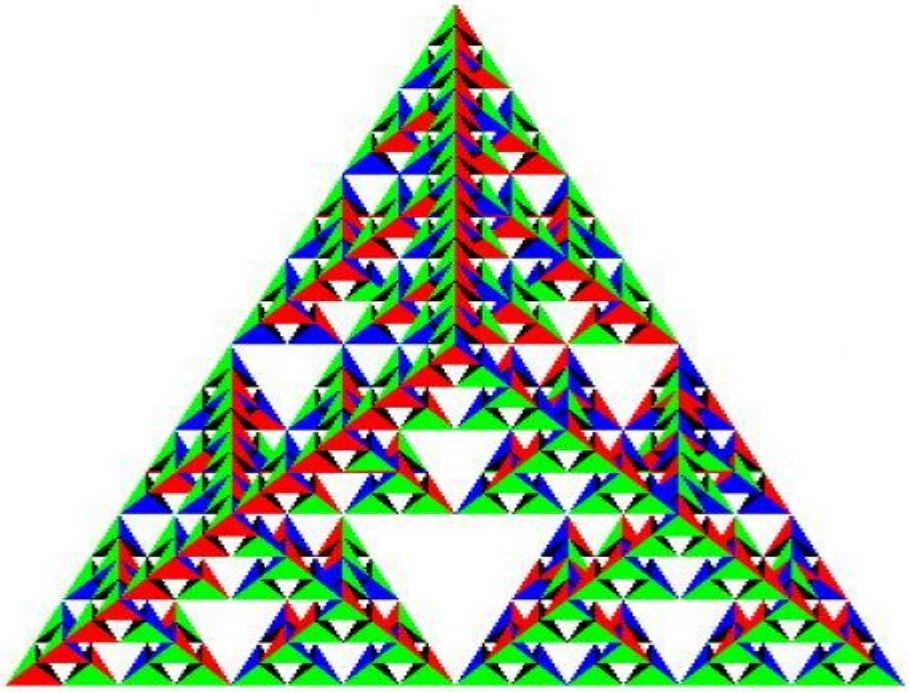


recursion



Sierpinski Gasket





Construction of Sierpinski Gasket - using random numbers

Possible form of our program might be:

```
main ( )
{
        initialize_the_system ();
        for (some_number_of_points )
        {
             pt = generate_a_point();
             display_the_point(pt);
        }
        cleanup()
}
```

Construction of Sierpinski Gasket using OpenGL

```
#include <GL/glut.h>
void main (int argc, char **argv) /* the main function */
{
    glutInit(&argc,argv);
    glutDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize (500,500);
    glutInitWindowPosition(0,0);
    glutCreateWindow("simple OpenGL example");
    glutDisplayFunc(display);
    myinit( );
    glutMainLoop( ); // Event Processing
}
```

Construction of Sierpinski Gasket using OpenGL

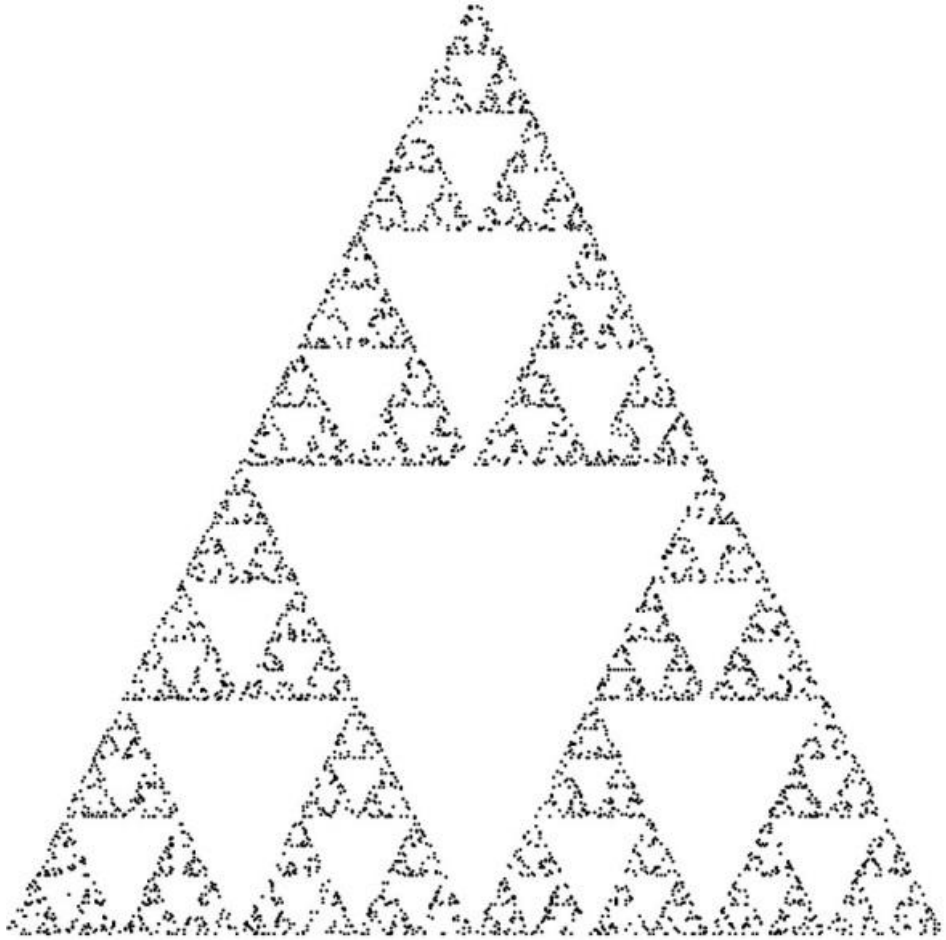
```
typedef GLfloat point2 [2];
/* defines a point as two dimensional array */

void display(void)
{    /* an arbitrary triangle */
     point2 vertices[3]= { {0.0,0.0}, {250.500}, {500.0,0.0}};
     static point2 p = {75.0, 50.0}; /* set to any initial point */
     int j,k;
     int rand( ); /* standard random number generator */
```

Construction of Sierpinski Gasket using OpenGL

```
for (k=0;k<5000;k++)
{    j = rand ( )%3; /* pick a random vertex from 0, 1, 2 */
     p[0] = (p[0]+ vertices[ j ][0])/2;
     p[1] = (p[1]+ vertices[ j ][1])/2;
     /* display the new point */
     glBegin(GL_POINTS);
     glVertex2fv(p);
     glEnd();
}
glFlush(); /* points are to be displayed as soon as possible */
}
```

Sierpinski Gasket - output



Construction of Sierpinski Gasket using OpenGL

- The function ***rand*** is a standard random-number generator that produces a new random integer each time that it is called.
- Modulus operator is used to reduce these random integers to the three integers 0, 1, and 2.
- The call to ***glFlush*** ensures that points are rendered to the screen as soon as possible. If it is left, the program works correctly, but in a busy or networked environment there may be noticeable delay.
- ***A complete program is not yet written.***

Basic OpenGL types

- **GLfloat** and **GLint**, are used rather than the C types, such as float and int.
- These types are defined in the header files and usually in the obvious way.
- For example, **#define GLfloat float**
- However, use of the OpenGL types allows additional flexibility for implementations.
- For example, suppose the floats are to be changed to doubles without altering existing application programs.

Basic OpenGL types

- Returning to the vertex function, if the user wants to work in 2-D with integers, then the form

  glVertex2i(GLint xi, GLint yi) is appropriate
- **glVertex3f(GLfloat x, GLfloat y, GLfloat z)** specifies a position in 3-D space using floating-point numbers.
- If an array is used to store the information for a 3-D vertex, **GLfloat vertex[3]** then

  glVertex3fv(vertex) can be used.

Geometric primitives

- Different numbers of vertices are required depending on the object.
- Any number of vertices can be grouped using the ***functions*** **glBegin** and **glEnd.**
- The argument of **glBegin** specifies the geometric type that the vertices define. Hence, a line segment can be specified by

```
glBegin(GL_LINES);
 glVertex2f(xl,yl);
 glVertex2f(x2,y2);
glEnd();
```